

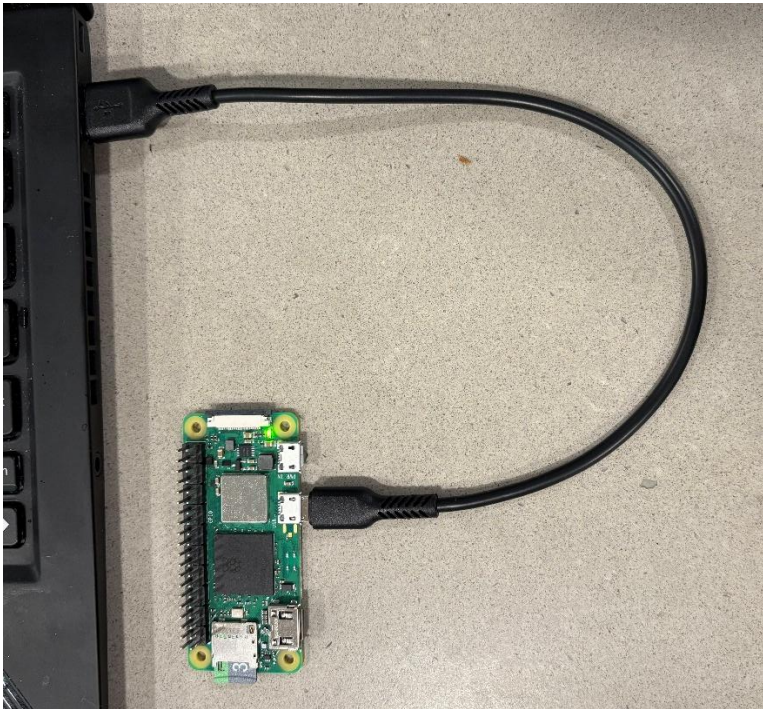
Contents

Getting Started	2
Connect the Raspberry Pi Zero 2 W	2
Connect to the Raspberry Pi using SSH	4
Using the Serial Console (UART)	5
Connect the USB to TTL Serial Cable	5
Windows	5
Mac	8
Login	8
Exercise 1: Scanning for Open Ports	9
Windows	9
Mac	10
Exercise 2: Brute Force the "root" User Password	11
Windows	11
Mac	11
Running Nmap	11
Exercise 3: Find and Brute Force Another User Account	12
Exercise 4: Capture and Observe Network Traffic	13
Exercise 5: Observe and Manipulate HTTP Traffic	17
Exercise 6: Check the Security State of Linux Packages	21
Exercise 7: Review AWS IoT Core Configuration	23
Exercise 8: Review Over-the-Air (OTA) Update Process	26
Exercise 9: Review Disk Encryption	28
Exercise 10: Tamper with the Bootloader	30
Raspberry Pi Boot Flow	31
Exercise 11: Counterfeiting the Device	32
Secure Boot	33
Security Modules	33
References	34
Prerequisites	34

Getting Started

Connect the Raspberry Pi Zero 2 W

1. Ensure the SD card is fully seated in the SD slot, then connect the micro-USB end of the black USB cable to the middle port as shown below. Connect the other end of the cable to a USB port on your computer. You should see a green LED light up and start blinking while the device boots.



2. After 1-2 minutes you should be able to open a Command Prompt (Windows) or Terminal (Mac) window and run `ping raspberrypi.local` to verify that the USB Ethernet connection between your computer and the Pi device is established.

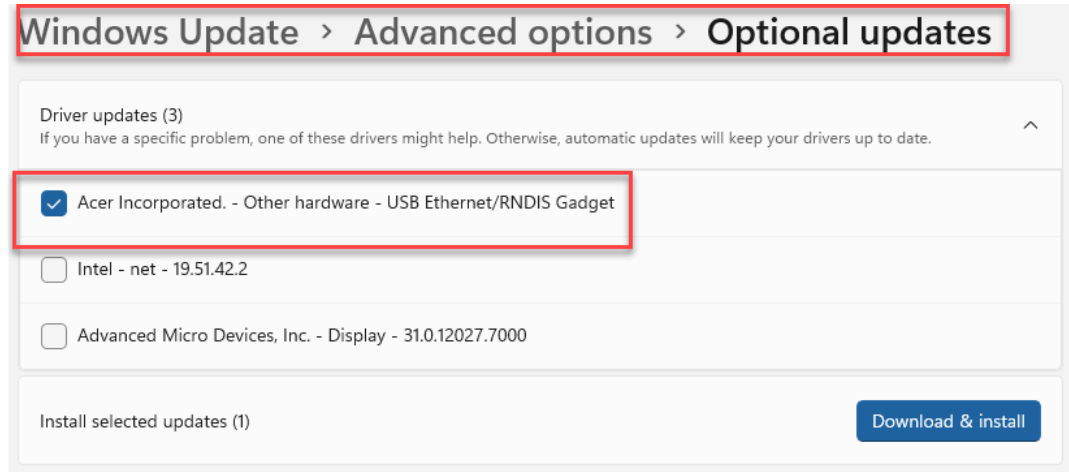
```
Command Prompt

C:\Users\Brice>ping raspberrypi.local

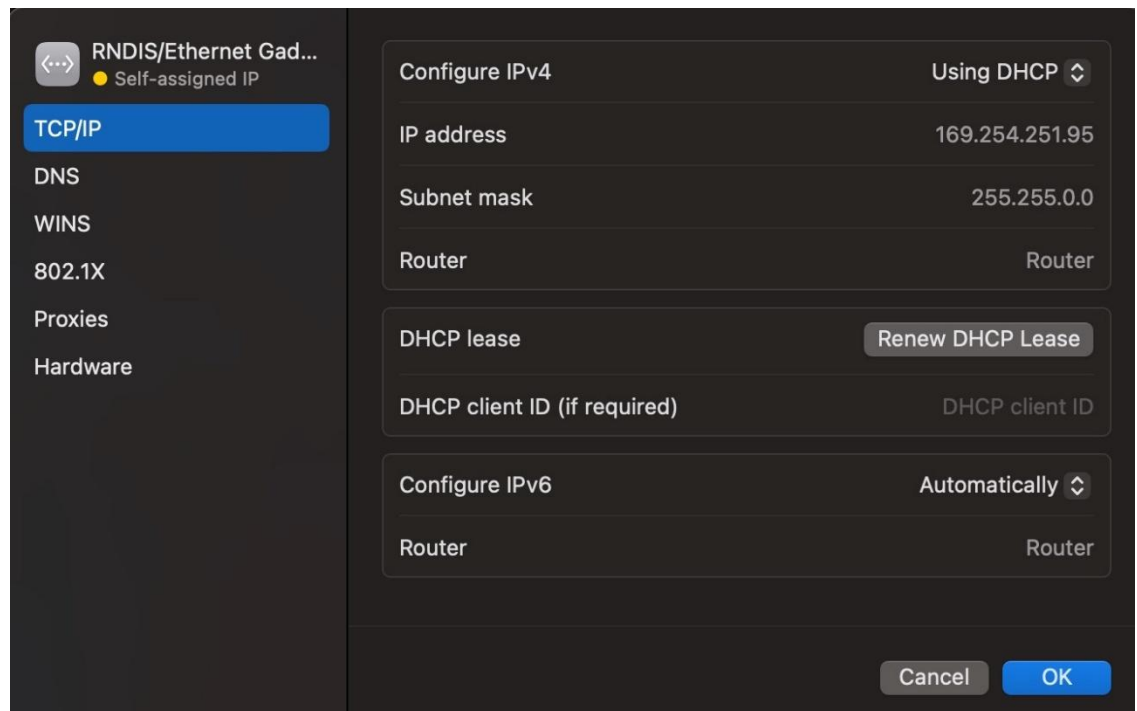
Pinging raspberrypi.local [fe80::d3f0:d086:eef9:8cab%23] with 32 bytes of data:
Reply from fe80::d3f0:d086:eef9:8cab%23: time<1ms
Reply from fe80::d3f0:d086:eef9:8cab%23: time<1ms
Reply from fe80::d3f0:d086:eef9:8cab%23: time=1ms
Reply from fe80::d3f0:d086:eef9:8cab%23: time<1ms

Ping statistics for fe80::d3f0:d086:eef9:8cab%23:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms
```

3. **Troubleshooting**
 - a. **Windows:** If there is no response when pingging `raspberrypi.local`, open Windows Update and ensure that all updates are installed. You may need to manually check for updates and potentially restart your computer for the USB Ethernet connection to be established. On Windows 11, be sure to check the Windows Update > Advanced Options > Optional page for a USB Ethernet/RNDIS Gadget driver update. See <https://joe.blog.freemansoft.com/2022/11/installing-rndis-driver-on-windows-11.html>

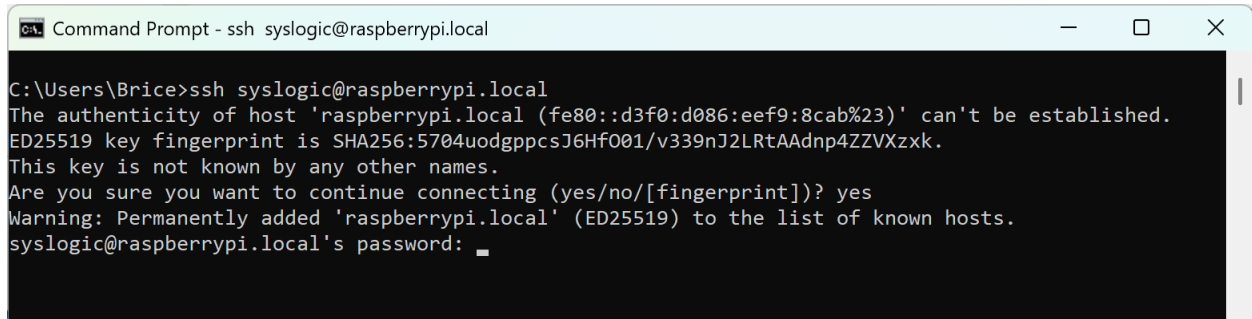


- b. **Mac:** If there is no response when pinging raspberrypi.local, open System Settings > Network and look for an **RNDIS/Ethernet Gadget** interface. Click on this interface and then click the **Details...** button. Try changing the setting on the TCP/IP blade from "Using DHCP" to **Manually**, then use an IP address of **169.254.251.1** and click the **OK** button. Then do these steps again to set it back to **Using DHCP**. Be sure to wait a minute or two before trying to ping the device again.



Connect to the Raspberry Pi using SSH

1. Open a Command Prompt (Windows) or Terminal (Mac) window and run the following command:
 - a. `ssh syslogic@raspberrypi.local`
2. The first time you connect, you will receive a warning that the SSH server public key is not known. Enter **yes** to continue connecting as shown below.

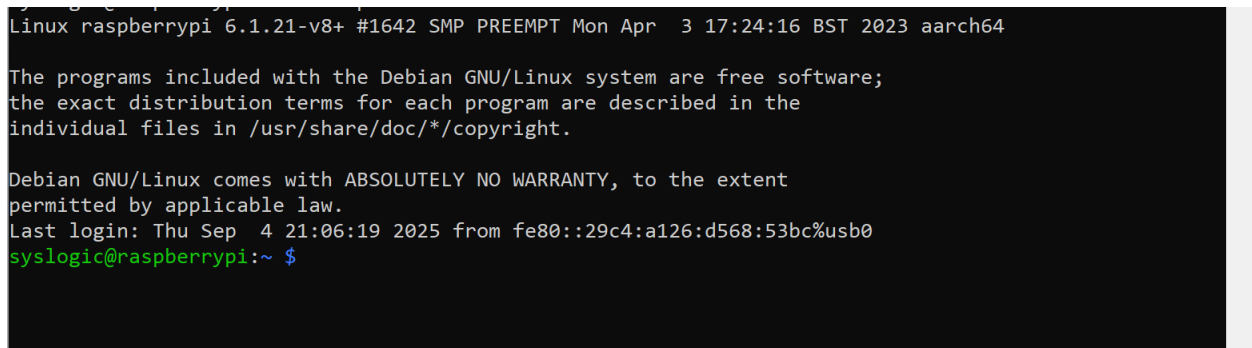


```

C:\Users\Brice>ssh syslogic@raspberrypi.local
The authenticity of host 'raspberrypi.local (fe80::d3f0:d086:eef9:8cab%23)' can't be established.
ED25519 key fingerprint is SHA256:5704uodgppcsJ6Hf001/v339nJ2LRtAAdnp4ZZVXzsk.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'raspberrypi.local' (ED25519) to the list of known hosts.
syslogic@raspberrypi.local's password: 

```

3. Enter the password of: **password123**
 - a. Note: you won't see the password characters as you type them.
4. After entering the password correctly, you should be presented with the information shown below and be successfully logged into the Raspberry Pi's Linux operating system.



```

Linux raspberrypi 6.1.21-v8+ #1642 SMP PREEMPT Mon Apr  3 17:24:16 BST 2023 aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

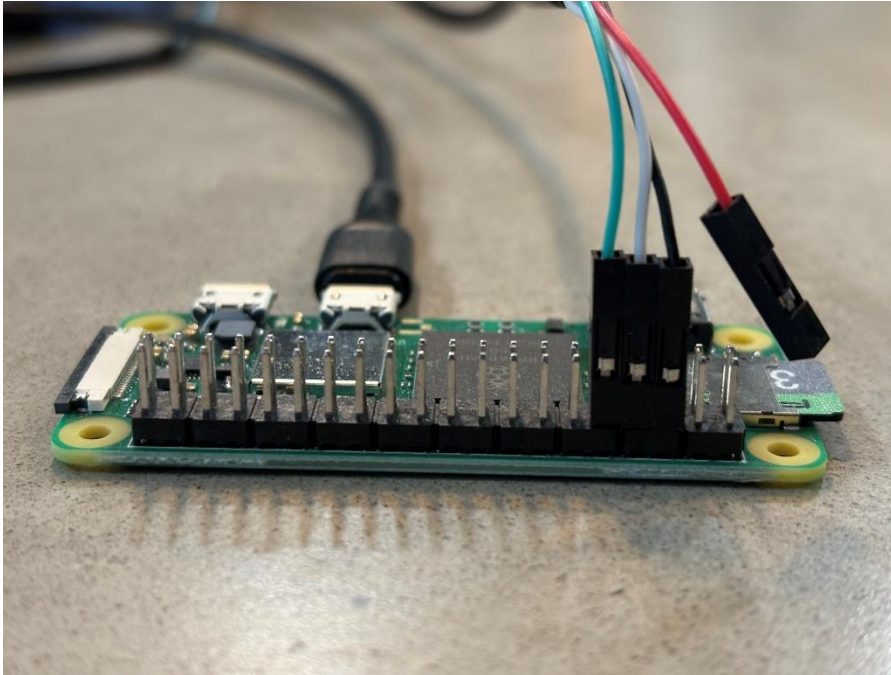
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Sep  4 21:06:19 2025 from fe80::29c4:a126:d568:53bc%usb0
syslogic@raspberrypi:~ $

```

Using the Serial Console (UART)

Connect the USB to TTL Serial Cable

1. Attach the black, white, and green connectors of the USB to TTL serial cable to pins 3-5 from the right when the Pi device is positioned so that the SD card is on the right. See the image below. Do not attach the red connector.



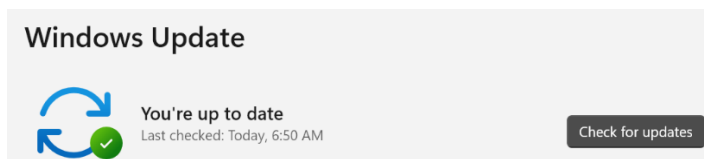
2. Connect the other blue end of the serial cable to a free USB port on your computer.

Windows

1. Open Device Manager. If you don't see a "Prolific USB-to-Serial Comm Port" item under Ports (COM & LPT), you will need to install a driver for the cable.

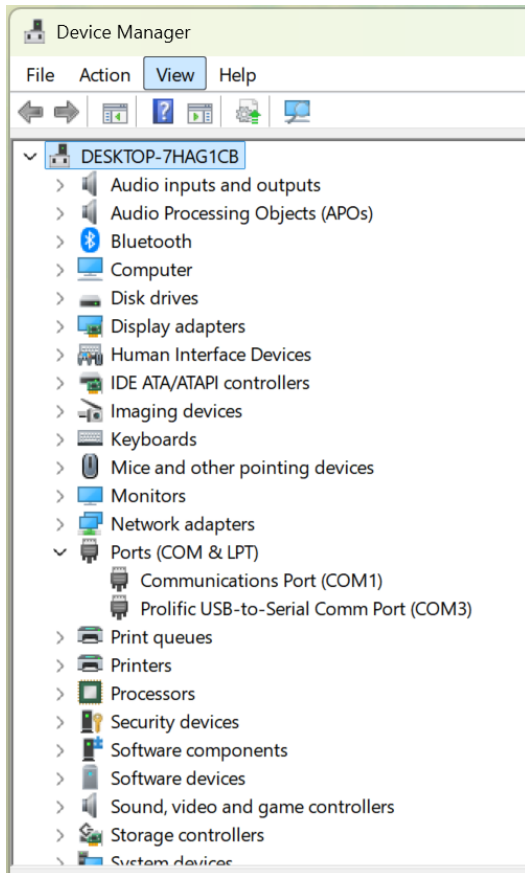
To install the PL2303 driver, you have two options:

- The first is to open Windows Update ("Check for updates" in the Windows Start menu), click the "Check for updates" button in the top right. Choose to install all applicable updates if given the option, but Windows will typically install the driver automatically without asking you to confirm.



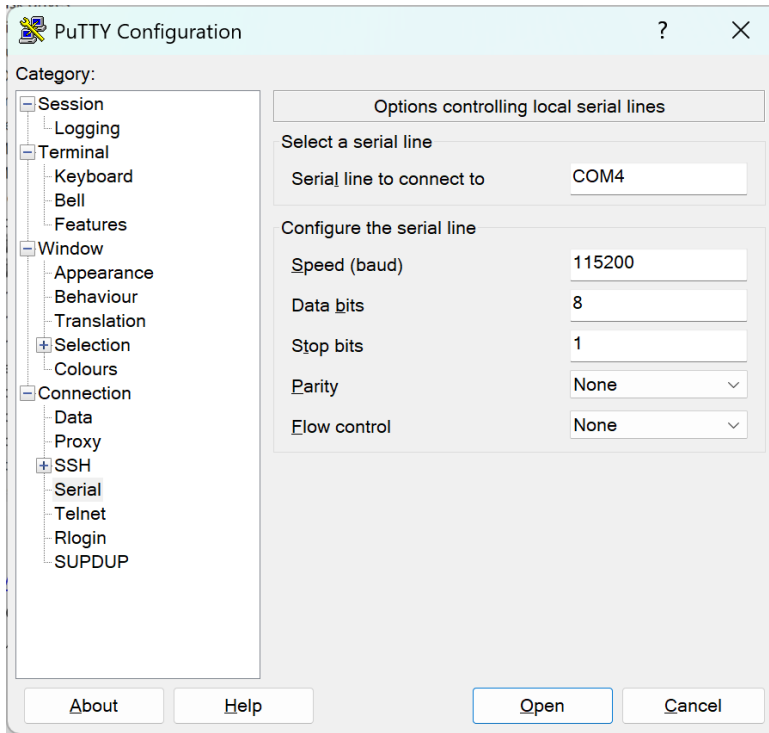
- The second option if the first one doesn't work is to manually download and install the Windows driver from <https://oemdrivers.com/usb-prolific-pl2303>

2. Once the driver is installed, you should see a new entry show under Ports (COM & LPT) showing "Prolific USB-to-Serial Comm Port (COMX)".
3. Make a note of the COM number used (3 in the screenshot shown below).

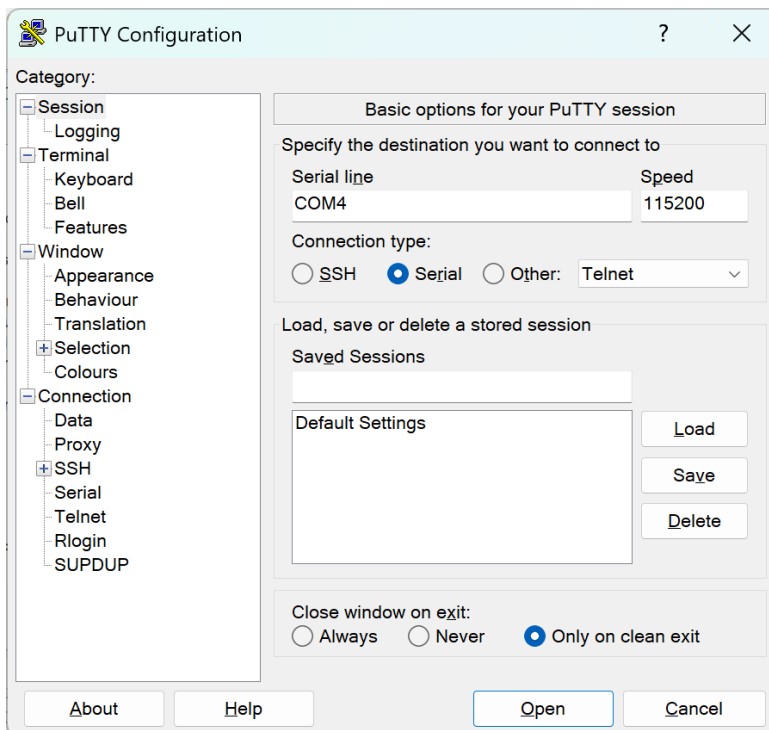


4. Open the PuTTY application if installed.
 - If not installed, you can download it from <https://www.chiark.greenend.org.uk/~sgtatham/putty/latest.html>. Choose the MSI (Windows Installer) option unless you do not have local administrator permission, otherwise download just the putty.exe binary file.

- Click the **Serial** category in the left panel under **Connection**. In the **Serial line to connect to** field, make sure it is set to the new COM port. Set **Speed** to **115200** and make sure **Flow control** is set to **None**.



- Click the **Session** category at the top of the left pane. Make sure **Serial line** is set correctly and that **Speed** is set to **115200**. Click the **Open** button at the bottom of the dialog window.

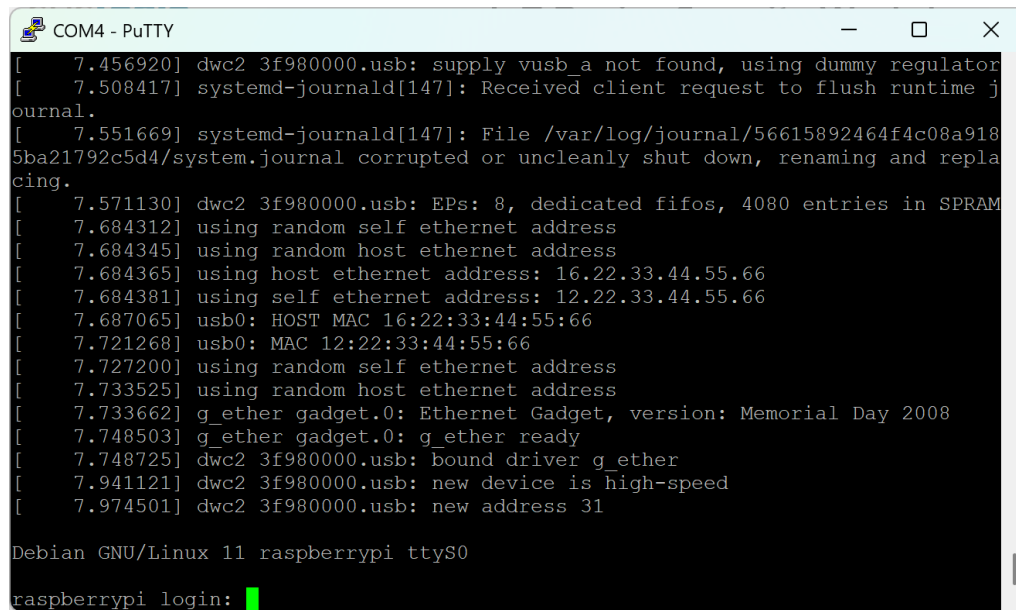


Mac

1. Install Minicom if not already installed:
 - a. `brew install minicom`
2. Run Minicom with the following parameters:
 - a. `minicom -b 115200 -D /dev/tty.usbserial-0001`

Login

1. The steps are the same whether you are using Putty on Windows or Minicom on Mac. If the serial UART connection is successful, you should see output displayed when the Raspberry Pi is booting, then see a login prompt.
2. If the window is blank, try hitting the Enter key to see if it is paused in the middle of the login process.



```
COM4 - PuTTY
[ 7.456920] dwc2 3f980000.usb: supply vusb_a not found, using dummy regulator
[ 7.508417] systemd-journald[147]: Received client request to flush runtime journal.
[ 7.551669] systemd-journald[147]: File /var/log/journal/56615892464f4c08a9185ba21792c5d4/system.journal corrupted or uncleanly shut down, renaming and replacing.
[ 7.571130] dwc2 3f980000.usb: EPs: 8, dedicated fifos, 4080 entries in SPRAM
[ 7.684312] using random self ethernet address
[ 7.684345] using random host ethernet address
[ 7.684365] using host ethernet address: 16.22.33.44.55.66
[ 7.684381] using self ethernet address: 12.22.33.44.55.66
[ 7.687065] usb0: HOST MAC 16:22:33:44:55:66
[ 7.721268] usb0: MAC 12:22:33:44:55:66
[ 7.727200] using random self ethernet address
[ 7.733525] using random host ethernet address
[ 7.733662] g_ether gadget.0: Ethernet Gadget, version: Memorial Day 2008
[ 7.748503] g_ether gadget.0: g_ether ready
[ 7.748725] dwc2 3f980000.usb: bound driver g_ether
[ 7.941121] dwc2 3f980000.usb: new device is high-speed
[ 7.974501] dwc2 3f980000.usb: new address 31

Debian GNU/Linux 11 raspberrypi ttyS0
raspberrypi login: █
```

3. Use the following credentials to log in:

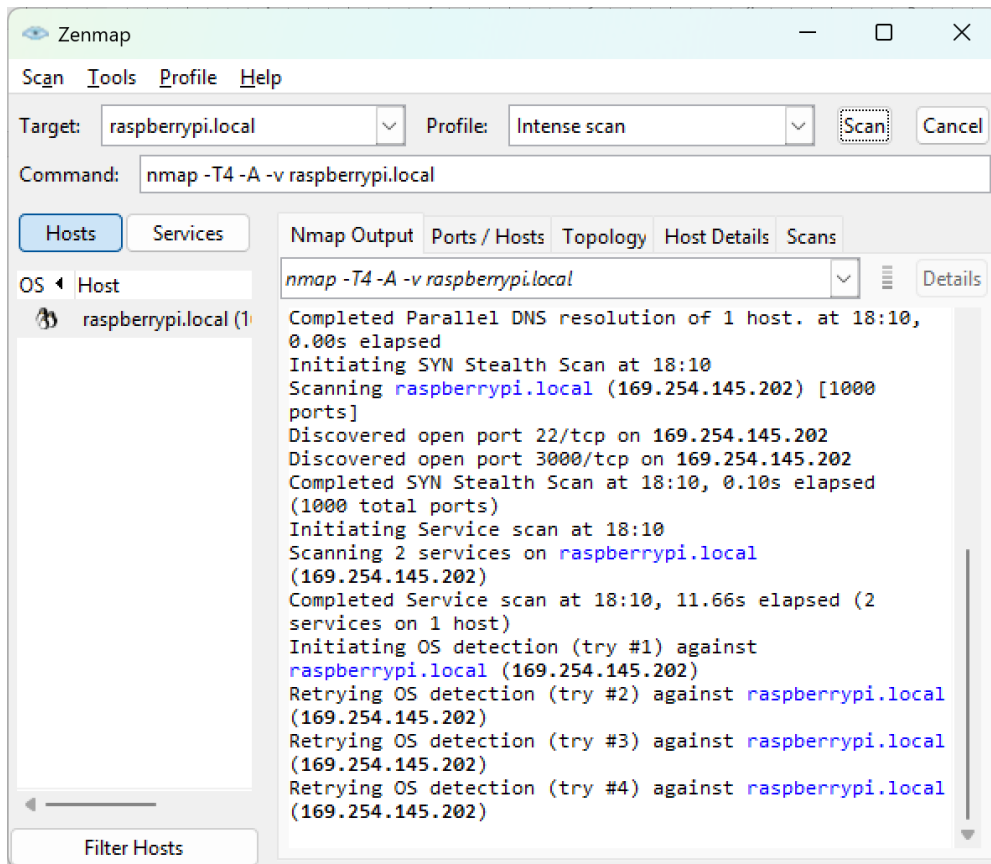
Username: **syslogic**

Password: **password123**

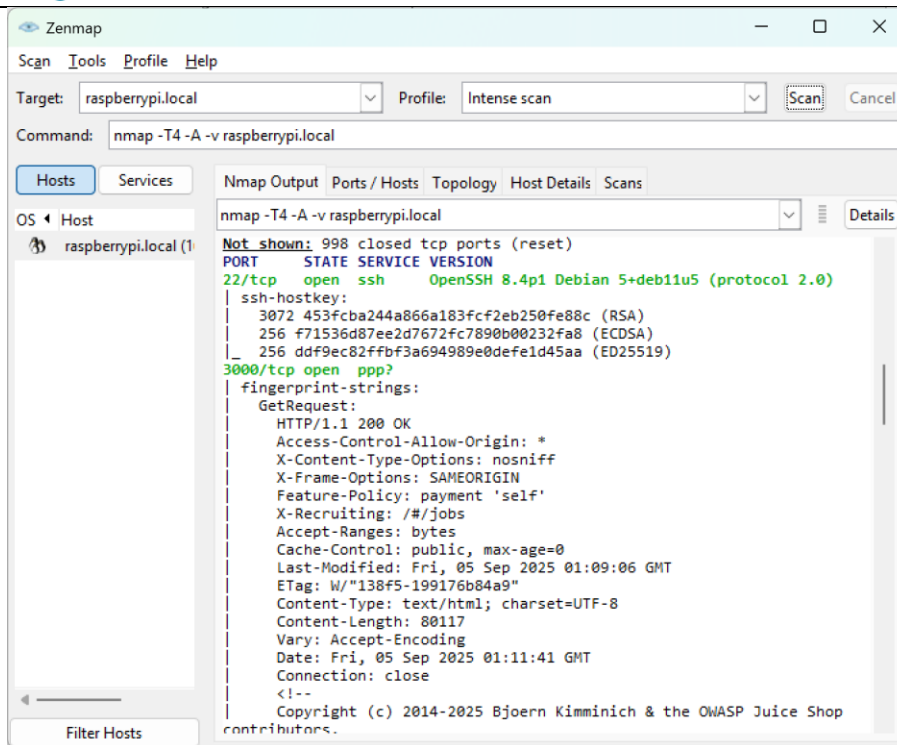
Exercise 1: Scanning for Open Ports

Windows

1. Open **Nmap – Zenmap GUI** from the Windows start menu if already installed. If not installed, and you have local administrator privileges, you can download and install it from <https://nmap.org/download.html#windows>
2. In the **Target** field, type **raspberrypi.local** Make sure the **Profile** drop down list is set to **Intense scan**, then click the **Scan** button.
3. The scan will take a minute or two during which you should see output information listed in the right-hand pane.



4. Once the scan is complete, you should be able to scroll up to see detailed information about the open ports detected. In the example below, port 22 is open and appears to be used by OpenSSH 8.4p1. Port 3000 is open and is returning what looks like an HTTP response.



5. Try browsing to <http://raspberryypi.local:3000> in a web browser to check if there is a web site or API running on port 3000.

Mac

1. If Nmap is not already installed, open a Terminal window and enter the following to install via Homebrew: `brew install nmap`
2. Once Nmap is installed, run the following to perform a scan of the device: `nmap -A -v raspberryypi.local`
 - a. This may take several minutes to complete.
3. Once the scan is complete, you should be able to scroll up to see detailed information about the open ports detected. In the example below, port 22 is open and appears to be used by OpenSSH 8.4p1. Port 3000 is open and is returning what looks like an HTTP response.
4. Try browsing to <http://raspberryypi.local:3000> in a web browser to check if there is a web site or API running on port 3000.

```
lindsey — -zsh — 80x39
Nmap scan report for raspberrypi.local (169.254.145.202)
Host is up (1.0s latency).
Other addresses for raspberrypi.local (not scanned): fe80::d3f0:d086:eef9:8cab
Not shown: 998 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.4p1 Debian 5+deb11u5 (protocol 2.0)
| ssh-hostkey:
|   3072 45:3f:cb:a2:44:a8:66:a1:83:fc:f2:eb:25:0f:e8:8c (RSA)
|   256  f7:15:36:d8:7e:e2:d7:67:2f:c7:89:0b:00:23:2f:a8 (ECDSA)
|_  256  dd:f9:ec:82:ff:bf:3a:69:49:89:e0:de:fe:1d:45:aa (ED25519)
3000/tcp  open  ppp?
| fingerprint-strings:
|   GetRequest:
|     HTTP/1.1 200 OK
|     Access-Control-Allow-Origin: *
|     X-Content-Type-Options: nosniff
|     X-Frame-Options: SAMEORIGIN
|     Feature-Policy: payment 'self'
|     X-Recruiting: /#/jobs
|     Accept-Ranges: bytes
|     Cache-Control: public, max-age=0
|     Last-Modified: Fri, 10 Oct 2025 04:13:22 GMT
|     ETag: W/"138f5-199cc52cdb0"
|     Content-Type: text/html; charset=UTF-8
|     Content-Length: 80117
|     Vary: Accept-Encoding
|     Date: Fri, 10 Oct 2025 04:57:53 GMT
|     Connection: close
|     <!--
|     Copyright (c) 2014-2025 Bjoern Kimminich & the OWASP Juice Shop contributors.
|
|     SPDX-License-Identifier: MIT
|     <!doctype html>
|     <html lang="en" data-beasties-container>
|     <head>
|     <meta charset="utf-8">
|     <title>OWASP Juice Shop</title>
|     <meta name="description" content="Probably the most modern and sophisticated insecure web application">
```

Exercise 2: Brute Force the “root” User Password

Windows

1. Create a new working folder. Example: C:\Users\<user>\Downloads\iotworkshop
2. Open **Command Prompt** from the Windows Start menu and change to the directory where nmap is installed:
 - a. `cd "C:\Program Files (x86)\Nmap"`

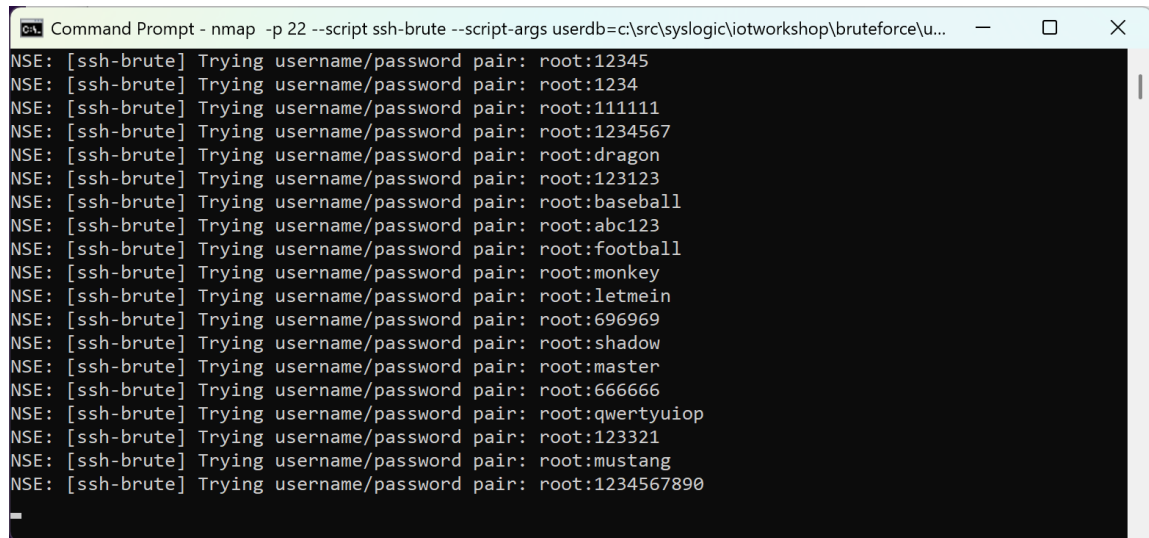
Mac

1. Create a new working folder. Example: ~\Downloads\iotworkshop
2. Open **Terminal**

Running Nmap

1. Create a file named **userlist.txt** in your working folder that contains a single line of text: **root**
2. Download the following password list file and place it in your working folder.
 - a. https://raw.githubusercontent.com/syslogicinc/iotworkshop/refs/heads/main/bruteforce/10_million_password_list_top_100.txt

3. Run the following command from your Command Prompt or Terminal window to start the brute force attack.
 - a. `nmap -p 22 --script ssh-brute --script-args userdb=<working-folder>/userlist.txt,passdb=<working-folder>/10_million_password_list_top_100.txt raspberrypi.local`
 - i. Replace <working-folder> with the actual path to your working folder.
 - b. This script should run for a minute or two testing each password against the root user.



```
Command Prompt - nmap -p 22 --script ssh-brute --script-args userdb=c:\src\syslogic\iotworkshop\bruteforce\u...
NSE: [ssh-brute] Trying username/password pair: root:12345
NSE: [ssh-brute] Trying username/password pair: root:1234
NSE: [ssh-brute] Trying username/password pair: root:111111
NSE: [ssh-brute] Trying username/password pair: root:1234567
NSE: [ssh-brute] Trying username/password pair: root:dragon
NSE: [ssh-brute] Trying username/password pair: root:123123
NSE: [ssh-brute] Trying username/password pair: root:baseball
NSE: [ssh-brute] Trying username/password pair: root:abc123
NSE: [ssh-brute] Trying username/password pair: root:football
NSE: [ssh-brute] Trying username/password pair: root:monkey
NSE: [ssh-brute] Trying username/password pair: root:letmein
NSE: [ssh-brute] Trying username/password pair: root:696969
NSE: [ssh-brute] Trying username/password pair: root:shadow
NSE: [ssh-brute] Trying username/password pair: root:master
NSE: [ssh-brute] Trying username/password pair: root:666666
NSE: [ssh-brute] Trying username/password pair: root:qwertyuiop
NSE: [ssh-brute] Trying username/password pair: root:123321
NSE: [ssh-brute] Trying username/password pair: root:mustang
NSE: [ssh-brute] Trying username/password pair: root:1234567890
```

- c. Once it completes, observe the output and see if you can determine the password for the root user.
- d. Try connecting via SSH to the Raspberry Pi as the root user to test that the password works.

Exercise 3: Find and Brute Force Another User Account

1. Connect to the Raspberry Pi via SSH or the serial console cable.
2. View the contents of the `/etc/shadow` file
 - a. `cat /etc/shadow`
3. There are quite a few user accounts defined. The rows that have “:!” directly after the username do not have a password associated with them and as a result, cannot be used to log into the device.
4. Look for a user account in this file that has a password that you don’t already know.

```

syslogic@raspberrypi: ~
root:$y$j9T$g.Gj71Gj6jwKb0TCwAP1$Th2NP8a25374qBdZ7SVvHki0RI0QzYATdhC9eJ.7/E8:20337:0:99999:7:::
daemon:*:20214:0:99999:7:::
bin:*:20214:0:99999:7:::
sys:*:20214:0:99999:7:::
sync:*:20214:0:99999:7:::
games:*:20214:0:99999:7:::
man:*:20214:0:99999:7:::
lp:*:20214:0:99999:7:::
mail:*:20214:0:99999:7:::
news:*:20214:0:99999:7:::
uucp:*:20214:0:99999:7:::
proxy:*:20214:0:99999:7:::
www-data:*:20214:0:99999:7:::
backup:*:20214:0:99999:7:::
list:*:20214:0:99999:7:::
irc:*:20214:0:99999:7:::
gnats:*:20214:0:99999:7:::
nobody:*:20214:0:99999:7:::
_apt:*:20214:0:99999:7:::
systemd-network:*:20214:0:99999:7:::

```

5. Update the userlist.txt file from Exercise #2 and replace **root** with the username you identified in step #4. Save your changes.
6. Using the technique in Exercise #2, perform another brute force attack to see if you can determine the password for this other user account.
 - a. `nmap -p 22 --script ssh-brute --script-args userdb=<working-folder>/userlist.txt,passdb=<working-folder>/10_million_password_list_top_100.txt raspberrypi.local`

Exercise 4: Capture and Observe Network Traffic

1. Open Wireshark if already installed.
 - a. If not installed, and you have local administrator privileges, you can download and install it from <https://www.wireshark.org/download.html>
2. You will need to select the network interface associated with the Raspberry Pi USB Ethernet. In the image below this interface is Ethernet 6.
 - a. To determine the correct interface to select, use one of the following methods:
 - i. Windows: open a Command Prompt window and run the `ipconfig` command. This may display quite a few interfaces. Look for the one that has an IPv4 address of 169.254.x.x.

```

Media State . . . . . : Media disconnected
Connection-specific DNS Suffix  . :

Ethernet adapter vEthernet (Default Switch):

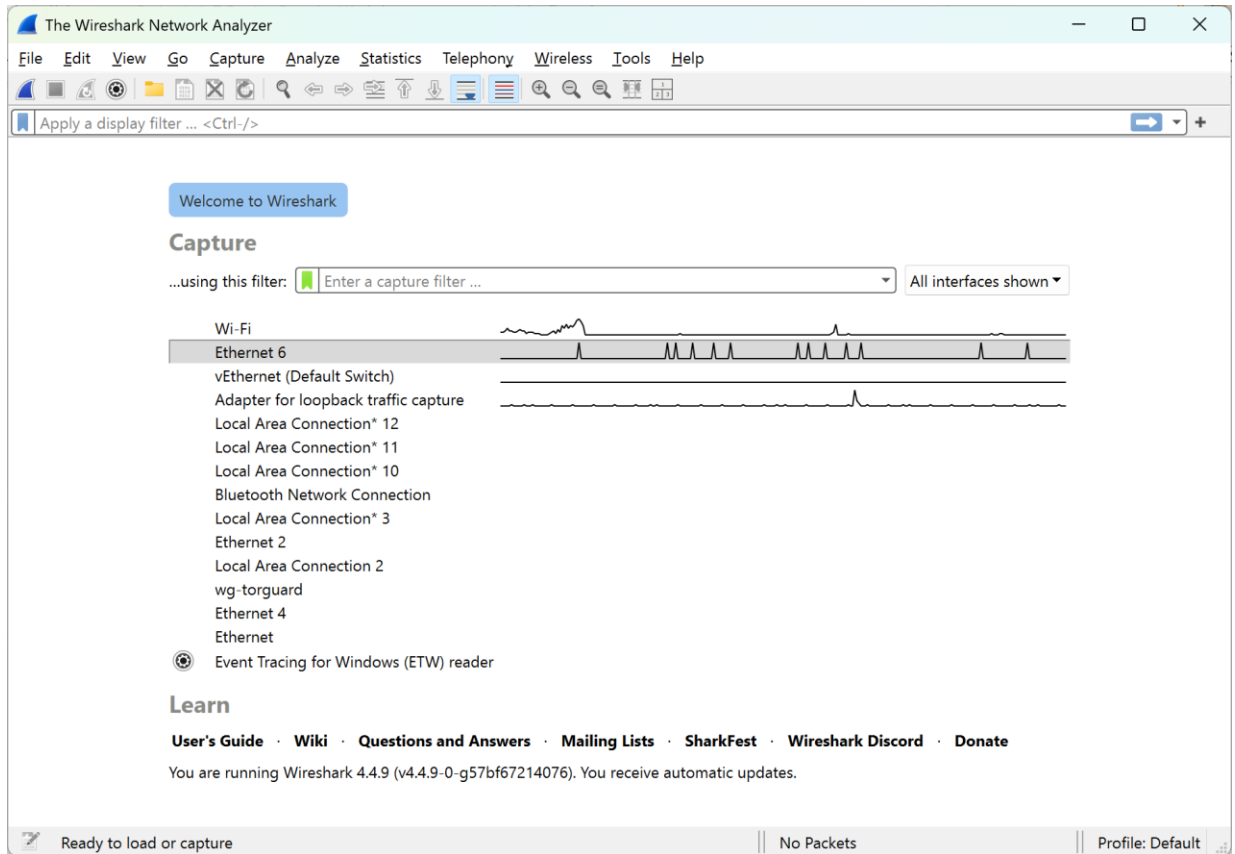
Connection-specific DNS Suffix  . :
Link-local IPv6 Address . . . . . : fe80::70b4:35bb:85ab:5834%6
IPv4 Address. . . . . : 172.26.144.1
Subnet Mask . . . . . : 255.255.240.0
Default Gateway . . . . . :

Ethernet adapter Ethernet 6:

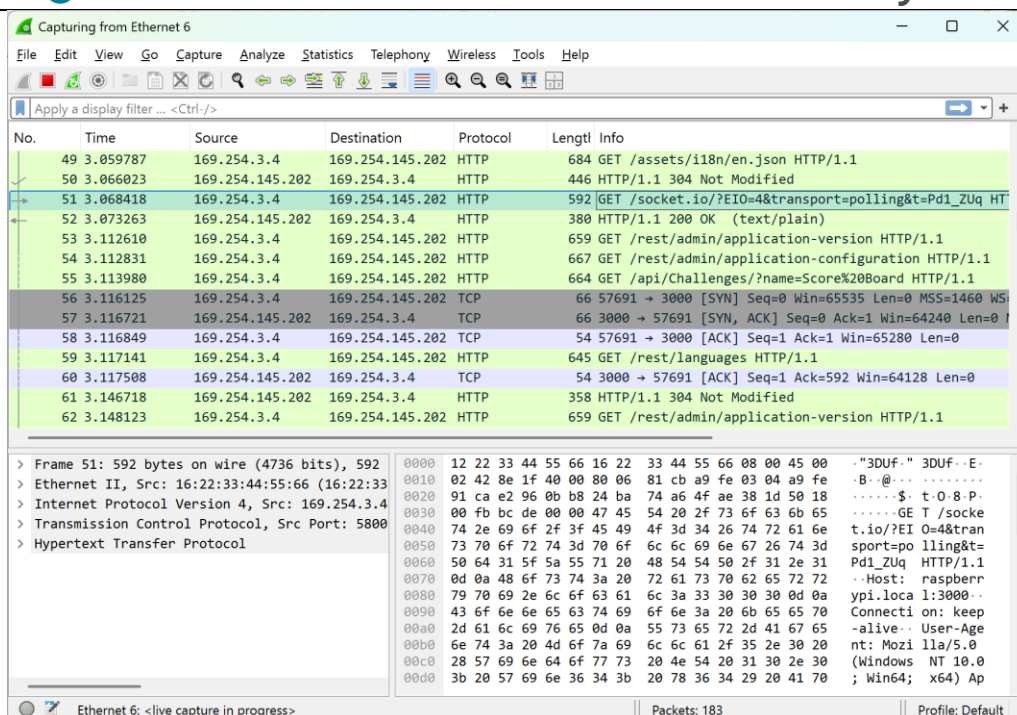
Connection-specific DNS Suffix  . :
Link-local IPv6 Address . . . . . : fe80::29c4:a126:d568:53bc%18
Autoconfiguration IPv4 Address. . : 169.254.3.4
Subnet Mask . . . . . : 255.255.0.0
Default Gateway . . . . . :

```

- ii. macOS: open a Terminal window and run the `ifconfig` command. This may display quite a few interfaces. Look for the one that has an IPv4 address of 169.254.x.x.



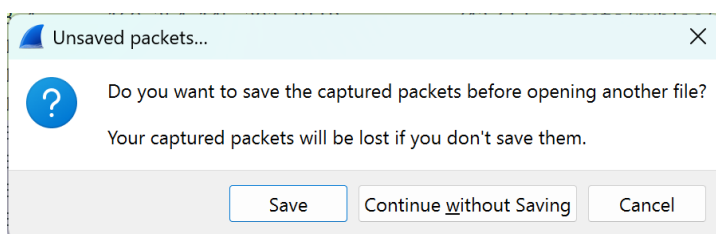
3. Once you select the correct network interface, double click on it, or right click on it and choose **Start capture**.
4. Open a browser and browse to <http://raspberrypi.local:3000>
5. In Wireshark you should now see the details of the network traffic between your laptop and the Raspberry Pi device.
6. Clicking on each item will allow you to view various details of the network packets, including protocol specific details with items like HTTP and WebSocket requests.



7. Click the red stop button on the Wireshark menu to stop the network traffic capture.
8. Connect to the Raspberry Pi via SSH using the instructions on page 2.
9. Run the command `sudo tcpdump -i wlan0 -w ~/tcpdump.log` to capture traffic sent over the Raspberry Pi's Wi-Fi interface.
10. Let it run for at least 10 seconds, then press Ctrl-C to stop the network traffic capture

```
syslogic@raspberrypi:~ $ sudo tcpdump -i wlan0 -w tcpdump.log
tcpdump: listening on wlan0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C22 packets captured
22 packets received by filter
0 packets dropped by kernel
syslogic@raspberrypi:~ $
```

11. Type `exit` and then hit **Enter** to exit out of the SSH connection.
12. Create a working folder if you have not already (see exercise #2)
1. In the Command Prompt/Terminal window enter the following command to copy the tcpdump.log file from the Raspberry Pi to your working folder over SSH.
 - b. `scp syslogic@raspberrypi.local:~/tcpdump.log <working-folder>`
 - i. Replace <working-folder> with the actual path to your working folder.
 - c. You will be prompted to enter the password for the syslogic account (password123)
13. In WireShark, select **File > Open** from the main menu, browse for the tcpdump.log file in your working folder, then click **Open**.
14. You will be prompted to save your captured packets from earlier. Choose **Continue without Saving** here.



- Observe the traffic sent/received from the Raspberry Pi device (IP address 192.168.101.x) and the cloud. In the highlighted packet below the destination IP address is 3.130.233.228. Notice also the mention of "MQ Telemetry Transport Protocol" at the bottom, or MQTT.

tcpdump.log

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.101.121	3.130.233.228	TLSv1.2	140	Application Data
2	0.078103	3.130.233.228	192.168.101.121	TCP	66	8883 → 51650 [ACK] Seq=1 Ack=75 Win=186 Len=0 TS
3	0.089339	3.130.233.228	192.168.101.121	TLSv1.2	99	Application Data
4	0.133532	192.168.101.121	3.130.233.228	TCP	66	51650 → 8883 [ACK] Seq=75 Ack=34 Win=1002 Len=0
5	0.221769	3.130.233.228	192.168.101.121	TLSv1.2	140	Application Data
6	0.221830	192.168.101.121	3.130.233.228	TCP	66	51650 → 8883 [ACK] Seq=75 Ack=108 Win=1002 Len=0
7	0.222178	192.168.101.121	3.130.233.228	TLSv1.2	99	Application Data
8	0.331664	3.130.233.228	192.168.101.121	TCP	66	8883 → 51650 [ACK] Seq=108 Ack=108 Win=186 Len=0
9	0.010659	192.168.101.121	3.130.233.228	TLSv1.2	140	Application Data
10	0.077666	3.130.233.228	192.168.101.121	TCP	66	8883 → 51650 [ACK] Seq=108 Ack=182 Win=186 Len=0
11	0.089338	3.130.233.228	192.168.101.121	TLSv1.2	99	Application Data
12	0.115291	3.130.233.228	192.168.101.121	TLSv1.2	140	Application Data
13	0.115384	192.168.101.121	3.130.233.228	TCP	66	51650 → 8883 [ACK] Seq=182 Ack=215 Win=1002 Len=
14	0.115776	192.168.101.121	3.130.233.228	TLSv1.2	99	Application Data

Frame 7: 99 bytes on wire (792 bits), 99 bytes captured (792 bits)

Ethernet II, Src: RaspberryPi_b2:46:e0 (2c:cf:67:b2:46:e0), Dst: ba:2f:00:00:55:c8 (08:00:27:ba:2f:00:00:00:55:c8)

Internet Protocol Version 4, Src: 192.168.101.121, Dst: 3.130.233.228

Transmission Control Protocol, Src Port: 51650, Dst Port: 8883, Seq: 75

Transport Layer Security

TLSv1.2 Record Layer: Application Data Protocol: MQ Telemetry Transport Protocol

Content Type: Application Data (23)

Version: TLS 1.2 (0x0303)

Length: 28

Encrypted Application Data: 00000000000034a4d81cf5ac1786e6da1b28

[Application Data Protocol: MQ Telemetry Transport Protocol]

Payload is encrypted application data (tls.app_data), 28 bytes

Packets: 22

Profile: Default

- Browse to <https://shodan.io> and enter the IP address 3.130.233.228. You will see this IP belongs to Amazon AWS and appears to be used with AWS Elastic Load Balancer (ELB).

3.130.233.228

Regular View Raw Data Timeline Whois

TAGS: cloud

LAST SEEN: 2025-10-05

General Information

Hostnames: ec2-3-130-233-228.us-east-2.compute.amazonaws.com

Domains: amazonaws.com

Cloud Provider: Amazon

Cloud Region: us-east-2

Cloud Service: EC2

Country: United States

City: Columbus

Organization: Amazon Technologies Inc.

ISP: Amazon.com, Inc.

ASN: AS16509

Open Ports

80 443

80 / TCP

1949896279

2025-10-05T20:40:27.406602

AWS ELB 2.0

301 Moved Permanently

HTTP/1.1 301 Moved Permanently

Server: awselb/2.0

Date: Sun, 05 Oct 2025 20:40:27 GMT

Content-Type: text/html

Content-Length: 134

Connection: keep-alive

Location: https://3.130.233.228:443/

443 / TCP

1102849523

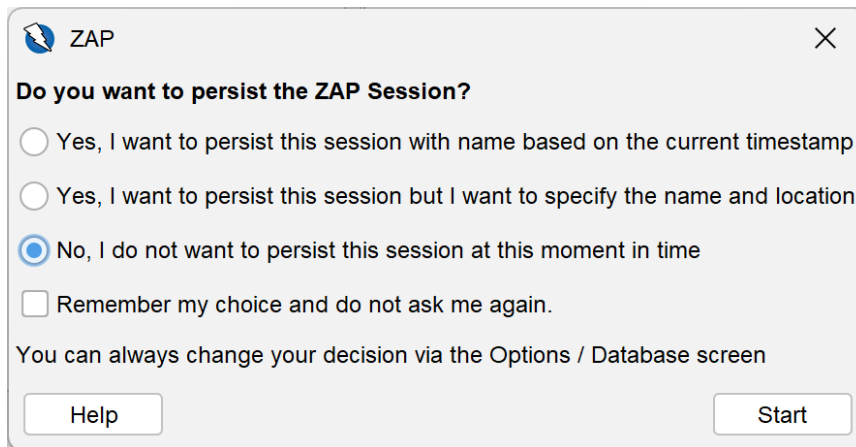
2025-10-05T20:49:48.575388

AWS ELB 2.0

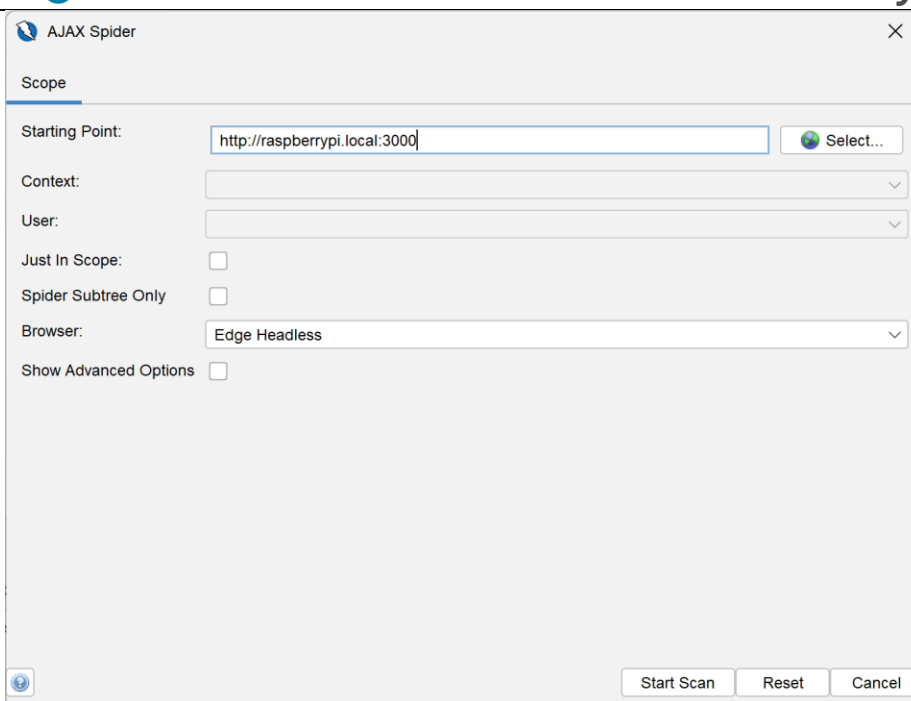
17. As a result of this reconnaissance, we now know that the Raspberry Pi device communicates with an AWS ELB endpoint using MQTT. We can make the reasonable assumption that this is communication with AWS IoT Core.
 - a. For this workshop, each Raspberry Pi device is configured to communicate with AWS IoT Core every 10 seconds using a pub/sub workflow to submit its MAC address.

Exercise 5: Observe and Manipulate HTTP Traffic

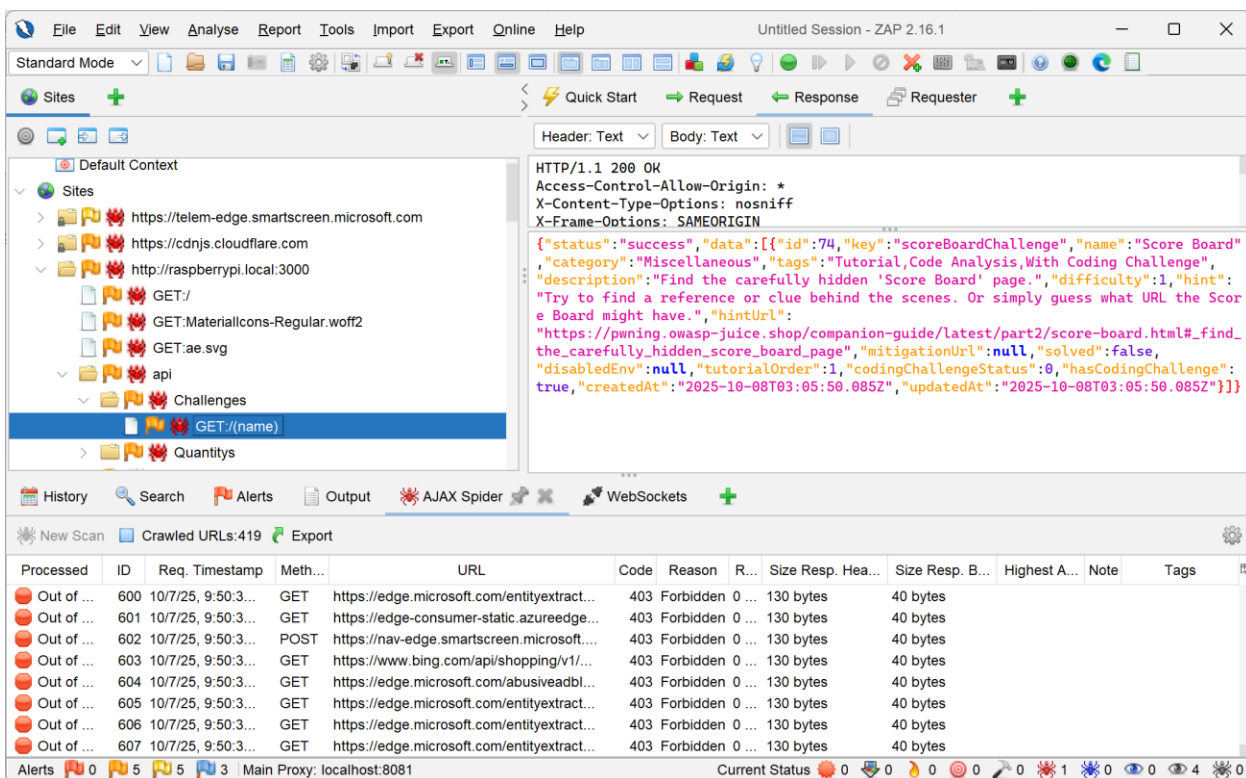
1. Open OWASP ZAP if already installed. If not installed, and you have local administrator privileges, you can download and install it from <https://www.zaproxy.org/download/>.
 - a. If you are on Windows and do not already have the Java JRE 17 or higher installed, use the winget option to include it: `winget install --id=ZAP.ZAP -e`
1. Choose the **No** option to not persist the session, then click the **Start** button



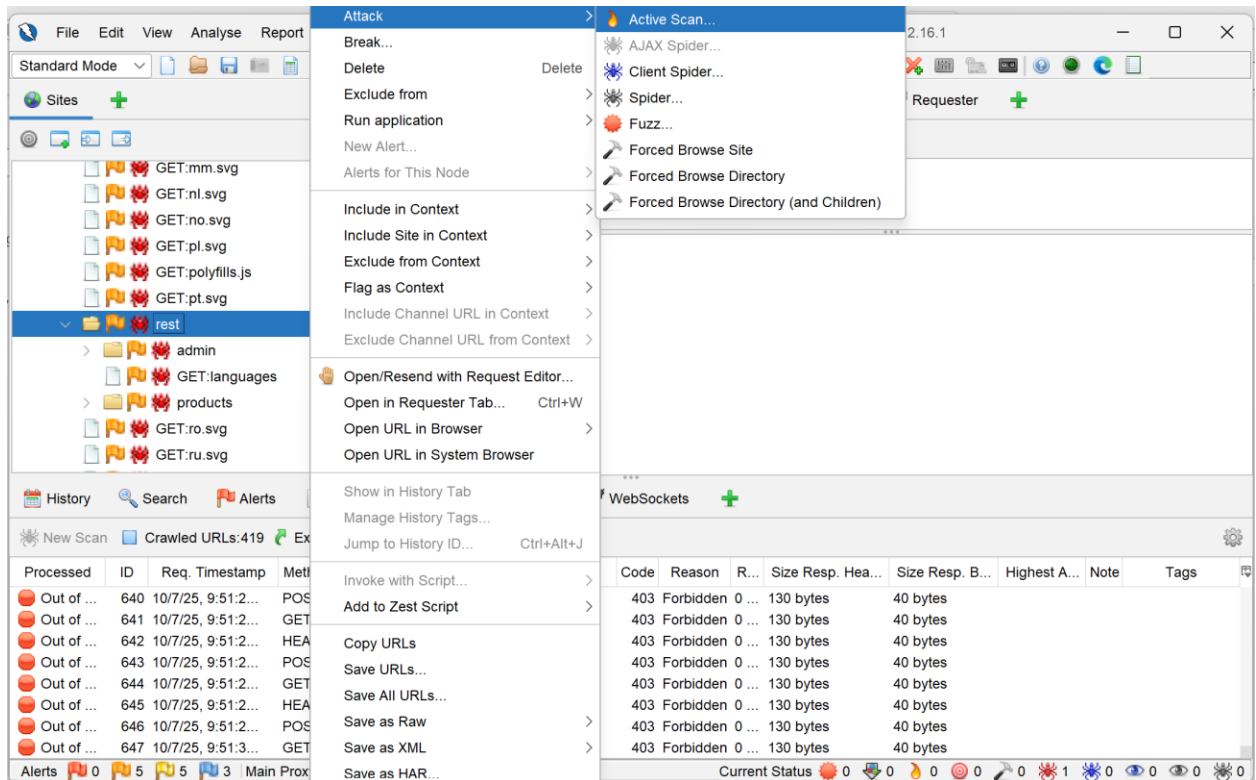
2. Click the **Tools > Ajax Spider...** option in the main menu, then enter **http://raspberrypi.local:3000** in the **Starting Point** field as shown below.
3. Select **Chrome Headless**, **Edge Headless**, or **Firefox Headless** in the Browser drop down list depending on the browser(s) you have installed. If you get an error message when trying to use one option, you may have better success selecting a different option.



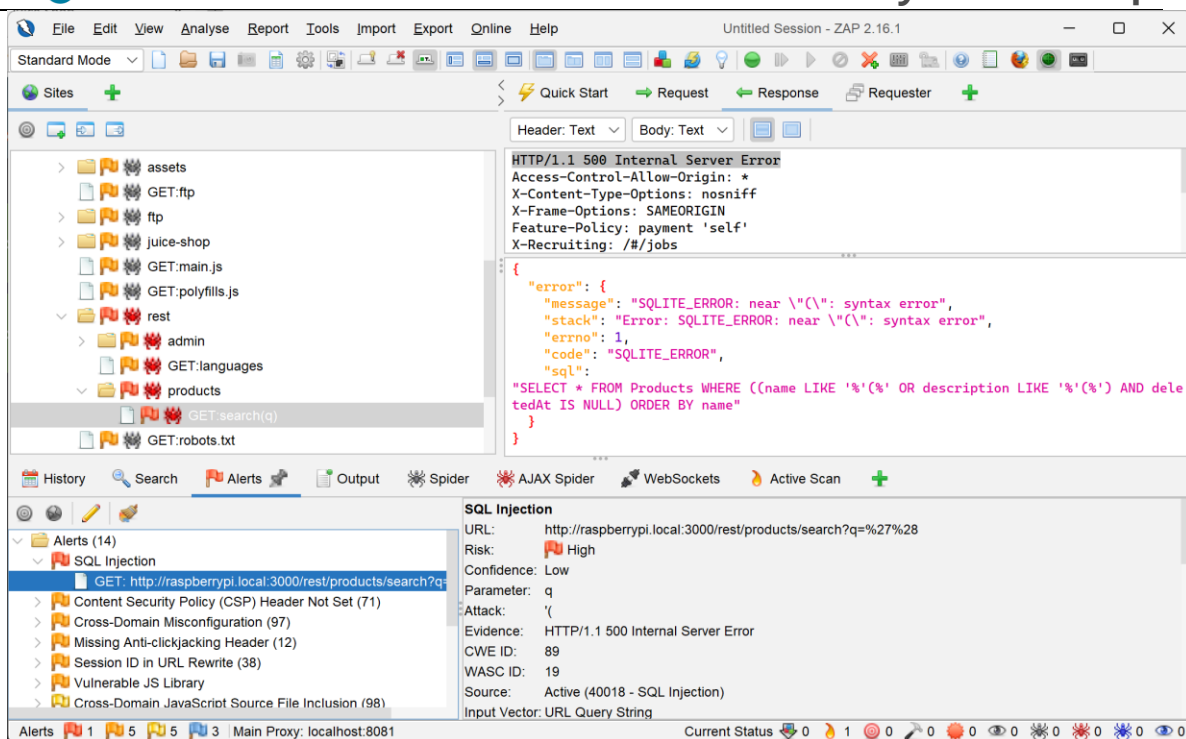
4. Click the **Start Scan** button.
5. Expand the Sites tree in the left-hand pane to view the site elements discovered by the spider scan.
6. Click on any request item and then view the HTTP request and response details in the upper right-hand pane as shown below.



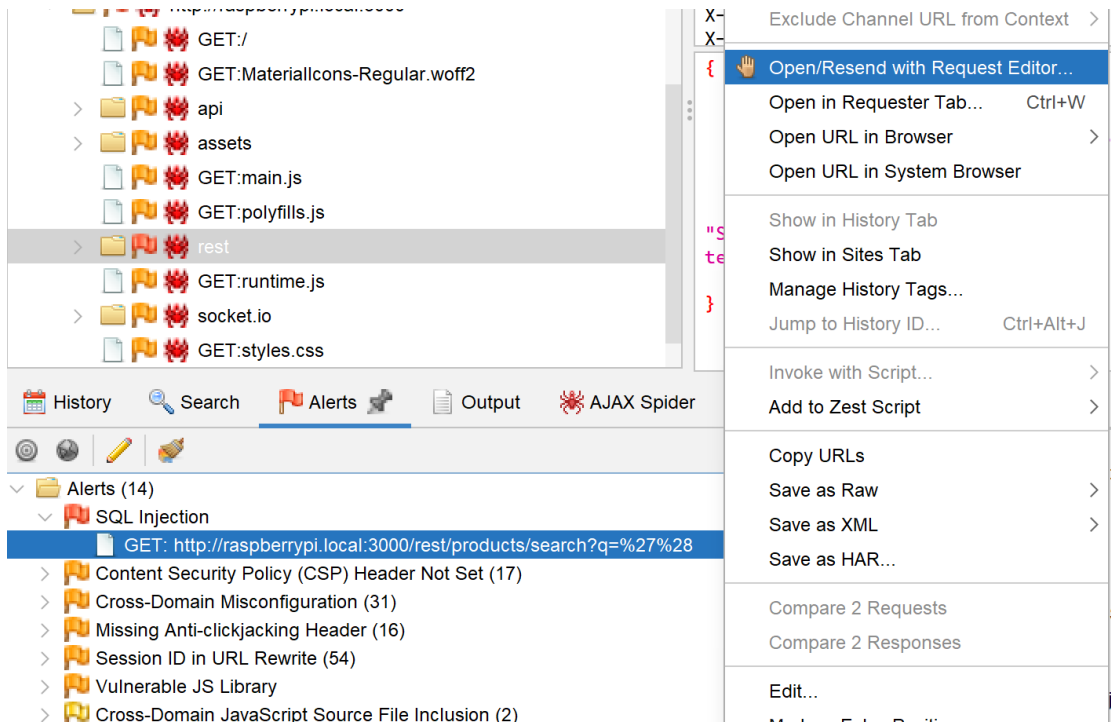
7. Right click on the `http://raspberrypi.local:3000 > rest` node in the Sites tree and choose **Attack > Active Scan..** from the context menu.

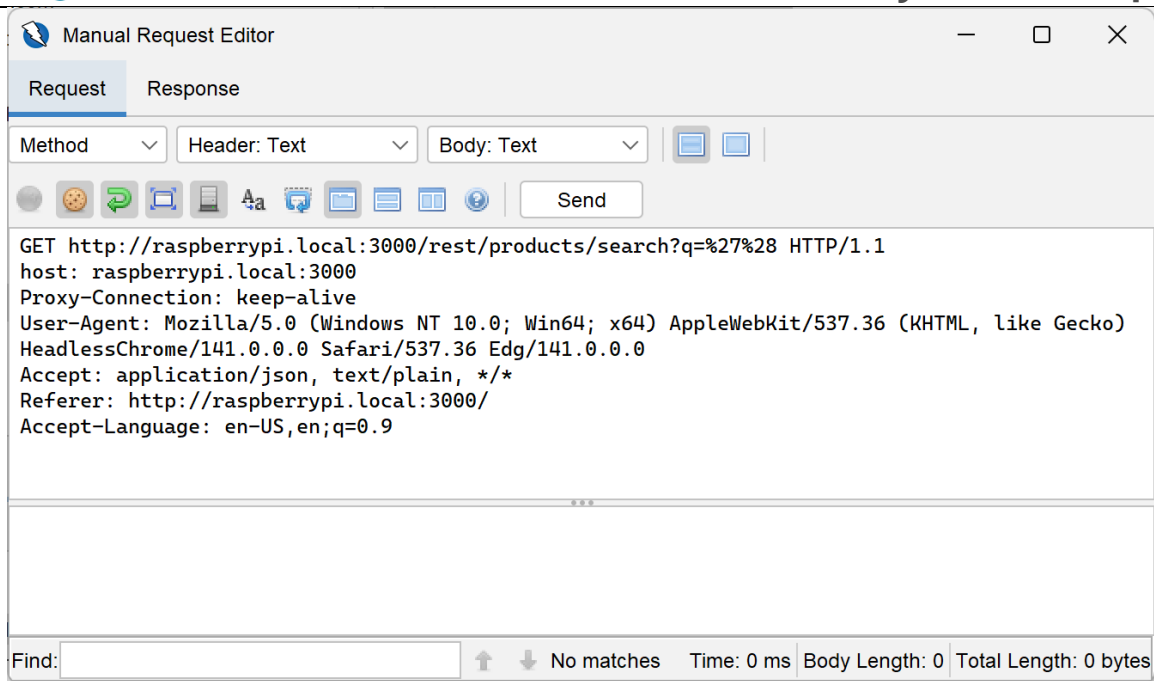


8. Click the **Start Scan** button to start the scan.
9. As the scan is running you can view issues discovered by the scan in the lower **Alerts** tab. Expanding this list and clicking on any specific alert will show you the corresponding HTTP request and response in the upper right-hand pane.

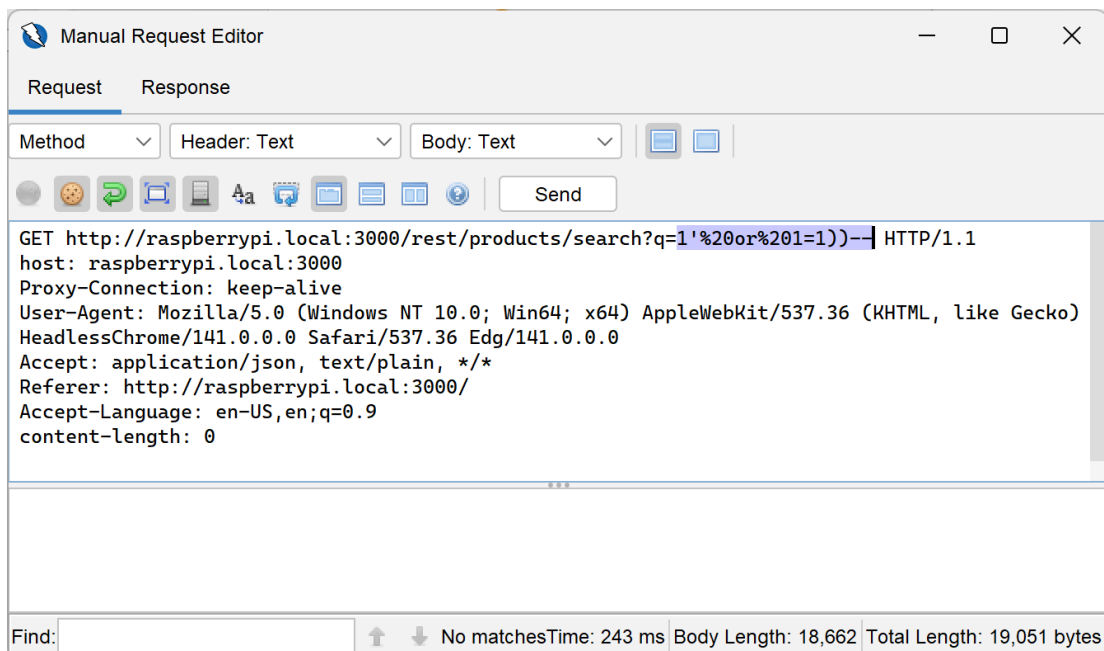


- Once the Active Scan is 40% complete or more, you should see a **SQL Injection** alert at the top of the Alerts list. Right click on this alert and choose **Open/Resend with Request Editor...** from the context menu.





11. You can change any part of the HTTP request on the Request tab, then click the Send button to send this HTTP request and review the results on the Response tab.
12. Try changing the value of the "q" querystring parameter to `1'%20or%201=1))--` and then click **Send** to see what happens.
 - a. What response do you get when sending this modified request?



13. What is the worst impact you could cause as an attacker by leveraging this vulnerability?

Exercise 6: Check the Security State of Linux Packages

1. Connect to the Raspberry Pi via SSH or the serial console cable.
2. Run the following command to update the local Linux package cache: `sudo apt update`


```

syslogic@raspberrypi: ~
syslogic@raspberrypi:~ $ sudo apt update
Hit:1 http://deb.debian.org/debian bullseye InRelease
Hit:2 http://deb.debian.org/debian bullseye-updates InRelease
Get:3 http://security.debian.org/debian-security bullseye-security InRelease [27.2 kB]
Get:4 http://archive.raspberrypi.org/debian bullseye InRelease [39.0 kB]
Get:5 http://security.debian.org/debian-security bullseye-security/main armhf Packages [403 kB]
Get:6 http://security.debian.org/debian-security bullseye-security/main arm64 Packages [406 kB]
Get:7 http://security.debian.org/debian-security bullseye-security/main Translation-en [275 kB]
Get:8 http://archive.raspberrypi.org/debian bullseye/main armhf Packages [330 kB]
Get:9 http://archive.raspberrypi.org/debian bullseye/main arm64 Packages [323 kB]
Fetched 1,803 kB in 10s (186 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
17 packages can be upgraded. Run 'apt list --upgradable' to see them.
syslogic@raspberrypi:~ $

```

3. Install the debsecan tool (this will take a minute): `sudo apt install -y debsecan`

```

syslogic@raspberrypi: ~
Preparing to unpack ../19-mailutils_1%3a3.10-3+b1_arm64.deb ...
Unpacking mailutils (1:3.10-3+b1) ...
Setting up mysql-common (5.8+1.0.7) ...
update-alternatives: using /etc/mysql/my.cnf.fallback to provide /etc/mysql/my.cnf (my.cnf) in auto mode
Setting up mariadb-common (1:10.5.29-0+deb11u1) ...
update-alternatives: using /etc/mysql/mariadb.cnf to provide /etc/mysql/my.cnf (my.cnf) in auto mode
Setting up libunbound8:arm64 (1.13.1-1+deb11u5) ...
Setting up libntlm0:arm64 (1.6-3) ...
Setting up libidn11:arm64 (1.33-3) ...
Setting up libfribidi0:arm64 (1.0.8-2+deb11u1) ...
Setting up mailutils-common (1:3.10-3) ...
Setting up libmariadb3:arm64 (1:10.5.29-0+deb11u1) ...
Setting up libgc1:arm64 (1:8.0.4-3) ...
Setting up libltdl7:arm64 (2.4.6-15) ...
Setting up debsecan (0.4.20.1) ...
Setting up exim4-config (4.94.2-7+deb11u4) ...

Progress: [ 77%] [#####.....]

```

4. Enter the command: `debsecan`


```
syslogic@raspberrypi: ~  
CVE-2025-4516 python3.9-dev  
CVE-2025-6069 python3.9-dev  
CVE-2025-8194 python3.9-dev  
CVE-2025-8291 python3.9-dev  
CVE-2024-5642 python3.9-minimal  
CVE-2025-4516 python3.9-minimal  
CVE-2025-6069 python3.9-minimal  
CVE-2025-8194 python3.9-minimal  
CVE-2025-8291 python3.9-minimal  
CVE-2022-29154 rsync  
CVE-2024-45310 runc  
CVE-2025-61984 ssh  
CVE-2025-61985 ssh  
CVE-2023-28486 sudo  
CVE-2023-28487 sudo  
CVE-2023-42465 sudo  
CVE-2021-3802 udisks2  
CVE-2025-8067 udisks2  
CVE-2025-53905 vim-common  
CVE-2025-53906 vim-common  
CVE-2025-53905 vim-tiny  
CVE-2025-53906 vim-tiny  
CVE-2021-31879 wget  
CVE-2024-10524 wget  
CVE-2025-53905 xxd  
CVE-2025-53906 xxd  
CVE-2023-45853 zlib1g  
CVE-2023-45853 zlib1g-dev  
syslogic@raspberrypi:~$
```

5. The debsecan tool analyzes the list of installed packages on the device and reports any vulnerabilities found. You can view the details of a specific vulnerability using the National Vulnerability Database at <https://nvd.nist.gov/vuln/search> and entering the corresponding CVE identifier.
6. Why are these vulnerable packages present? Many Linux systems are configured to automatically apply security fixes to address this using the unattended-upgrades package.
7. You can run the commands below to check if unattended-upgrades is installed on a system. For the workshop Raspberry Pi it is not installed. See <https://chrisapproved.com/blog/raspberry-pi-hardening.html#configure-automatic-updates> for more information about how to install and configure unattended-upgrades on the Raspberry Pi.

```
syslogic@raspberrypi: ~  
syslogic@raspberrypi:~$ sudo dpkg-query -f='${Package} ${Version} ${Architecture}\n' -W | grep unattended-upgrades  
syslogic@raspberrypi:~$ ls /var/log/unattended-upgrades/unattended-upgrades.log  
ls: cannot access '/var/log/unattended-upgrades/unattended-upgrades.log': No such file or directory  
syslogic@raspberrypi:~$
```

Exercise 7: Review AWS IoT Core Configuration

1. Connect to the Raspberry Pi via SSH or the serial console cable.
2. To view the list of background systemd services, run the command `sudo systemctl status` and scroll through the results.

```

syslogic@raspberrypi: ~
-ssh.service
  ↳477 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
-awsiot.service
  ↳611 /bin/bash /home/syslogic/aws-iot-device-sdk-python-v2/start.sh
  ↳618 python3 /home/syslogic/aws-iot-device-sdk-python-v2/samples/pubsub.py --
-hciuart.service
  ↳554 /usr/bin/hciattach /dev/serial1 bcm43xx 3000000 flow - b8:27:eb:a0:af:71
-rsyslog.service
  ↳364 /usr/sbin/rsyslogd -n -iNONE
-dhcpd.service
  ↳467 wpa_supplicant -B -c/etc/wpa_supplicant/wpa_supplicant.conf -iwlan0
  ↳610 /usr/sbin/dhcpd -w -q
-rng-tools-debian.service
  ↳440 /usr/sbin/rngd -r /dev/hwrng
-dbus.service
  ↳345 /usr/bin/dbus-daemon --system --address=systemd: --nofork --nopidfile --
-systemd-timesyncd.service
  ↳265 /lib/systemd/systemd-timesyncd
-system-getty.slice
  ↳getty@tty1.service
    ↳620 /sbin/agetty -o -p -- \u --noclear tty1 linux
-juiceshop.service
  ↳351 /bin/bash /root/start-juiceshop.sh
  ↳384 docker run --rm -p 3000:3000 bkimminich/juice-shop
-avahi-daemon.service
  ↳342 avahi-daemon: running [raspberrypi.local]
  ↳365 avahi-daemon: chroot helper
-systemd-logind.service
  ↳373 /lib/systemd/systemd-logind
lines 50-78/78 (END)

```

3. Notice that one of the services is named "awsiot.service" and that it launches a bash script which in return launches a Python script in the /home/syslogic/aws-iot-device-sdk-python-v2/samples folder.
4. Process Ctrl-C (or Command-C) to exit the systemctl status view.
5. Next, let's review the contents of the start.sh script by running this command: `cat /home/syslogic/aws-iot-device-sdk-python-v2/start.sh`

```

syslogic@raspberrypi:~ $ cat /home/syslogic/aws-iot-device-sdk-python-v2/start.sh
#!/bin/bash
# run pub/sub sample app
serial_number=$(cat /proc/cpuinfo | grep Serial | cut -d ' ' -f 2)
printf "\nRunning pub/sub sample application...\n"
python3 ~/aws-iot-device-sdk-python-v2/samples/pubsub.py --endpoint a3tdy1i9d7te5n-ats.iot.us-east-2.amazonaws.com --ca_file /opt/certs/awsiot/root-CA.crt --cert /opt/certs/awsiot/cert.pem --key /opt/certs/awsiot/private.key --client_id basicPubSub --topic sdk/test/python --count 0 --message "$serial_number"

```

6. Observe the references to certificates and a private key file in the /opt/certs/awsiot folder. Review the files permissions of this folder and its contents by running `ls -la /opt/certs/awsiot`

```

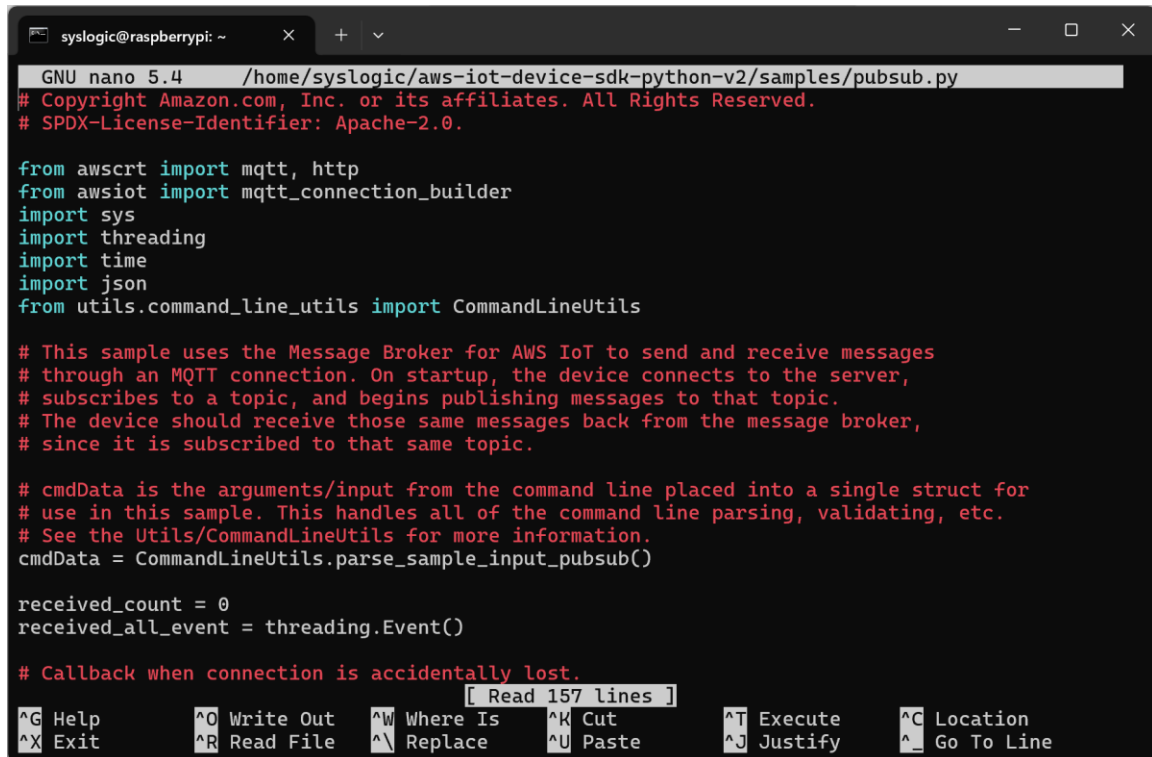
syslogic@raspberrypi:~ $ ls -la /opt/certs/awsiot
total 20
drwxr-xr-x 2 root root 4096 Aug 23 17:22 .
drwxr-xr-x 3 root root 4096 Aug 23 17:21 ..
-rw-r--r-- 1 root root 1220 Aug 23 17:21 cert.pem
-rw-r--r-- 1 root root 1679 Aug 23 17:22 private.key
-rw-r--r-- 1 root root 1188 Aug 23 17:22 root-CA.crt

```

7. We can see in the output above that the contents of these files are visible to all Linux users, not just the "syslogic" account. AWS recommends the following file permissions (see

<https://github.com/aws-labs/aws-iot-device-client/blob/main/docs/PERMISSIONS.md#file-and-directory-permissions>):

- a. Folder containing keys and certificates: 700 (rwx-----)
 - b. Private key files: 600 (rw-----)
 - c. Public certificate and root certificate authority files: 644 (rw-r--r--)
8. Next, view the contents of the pubsub.py file using nano for better readability: `nano /home/syslogic/aws-iot-device-sdk-python-v2/samples/pubsub.py`



```

syslogic@raspberrypi: ~
GNU nano 5.4 /home/syslogic/aws-iot-device-sdk-python-v2/samples/pubsub.py
# Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
# SPDX-License-Identifier: Apache-2.0.

from awscli import mqtt, http
from awscli import mqtt_connection_builder
import sys
import threading
import time
import json
from utils.command_line_utils import CommandLineUtils

# This sample uses the Message Broker for AWS IoT to send and receive messages
# through an MQTT connection. On startup, the device connects to the server,
# subscribes to a topic, and begins publishing messages to that topic.
# The device should receive those same messages back from the message broker,
# since it is subscribed to that same topic.

# cmdData is the arguments/input from the command line placed into a single struct for
# use in this sample. This handles all of the command line parsing, validating, etc.
# See the Utils/CommandLineUtils for more information.
cmdData = CommandLineUtils.parse_sample_input_pubsub()

received_count = 0
received_all_event = threading.Event()

# Callback when connection is accidentally lost.
[ Read 157 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^_ Replace   ^U Paste     ^J Justify   ^_ Go To Line

```

9. Scroll down to the section that starts with the comment “# Create a MQTT connection from the command line data”.

```

syslogic@raspberrypi: ~
GNU nano 5.4 /home/syslogic/aws-iot-device-sdk-python-v2/samples/pubsub.py
host_name=cmdData.input_proxy_host,
port=cmdData.input_proxy_port)

# Create a MQTT connection from the command line data
mqtt_connection = mqtt_connection_builder.mtls_from_path(
    endpoint=cmdData.input_endpoint,
    port=cmdData.input_port,
    cert_filepath=cmdData.input_cert,
    pri_key_filepath=cmdData.input_key,
    #ca_filepath=cmdData.input_ca,
    on_connection_interrupted=on_connection_interrupted,
    on_connection_resumed=on_connection_resumed,
    client_id=cmdData.input_clientId,
    clean_session=False,
    keep_alive_secs=30,
    http_proxy_options=proxy_options,
    on_connection_success=on_connection_success,
    on_connection_failure=on_connection_failure,
    on_connection_closed=on_connection_closed)

if not cmdData.input_is_ci:
    print(f"Connecting to {cmdData.input_endpoint} with client ID '{cmdData.input_clientId}'")
else:
    print("Connecting to endpoint with client ID")
connect_future = mqtt_connection.connect()

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
  
```

10. Notice that the "ca_filepath" parameter is commented out. When this is not set the mqtt_connection_builder uses the default trust store (see https://aws.github.io/aws-iot-device-sdk-python-v2/awsiot/mqtt_connection_builder.html). For Python versions less than 3.10, the "certifi" library is used for the trust store, which bundles Mozilla's CA certificates for HTTPS verification. The Raspberry Pi is using Python 3.9.2 (python --version).
11. For the Raspberry Pi device the certifi certificate trust can be found at /etc/ssl/certs/ca-certificates.crt. If you view the contents of this file (cat /etc/ssl/certs/ca-certificates.crt) you will see that this contains many root CA certificates.
 - The concern with using the default trust store is that it becomes more likely for an attacker to be able to generate a trusted TLS certificate for the AWS endpoint (a3tdy1i9d7te5n-ats.iot.us-east-2.amazonaws.com) by taking advantage of one compromised certificate authority (CA) from a large list of trusted ones. Using a rogue (but still trusted) TLS certificate combined with DNS spoofing could allow an attacker to intercept AWS IoT Core traffic sent between the device and cloud.

Exercise 8: Review Over-the-Air (OTA) Update Process

1. Connect to the Raspberry Pi via SSH or the serial console cable.
2. Run the command `ls /etc/systemd/system` to view the list of systemd services defined. Notice that one of them is named **ota.service**.
3. View the contents of the ota service definition: `cat /etc/systemd/system/ota.service`. You should see a reference to a bash script file in /opt/ota/ota.sh.
4. List the contents of the /opt/ota folder: `ls -la /opt/ota`

```

syslogic@raspberrypi:~ $ ls /etc/systemd/system
awsiot.service          dhcpcd.service.d      rc-local.service.d
bluetooth.target.wants  getty.target.wants    reboot.target.wants
dbus-fi.w1.wpa_supplicant1.service  getty@tty1.service.d  remote-fs.target.wants
dbus-org.bluez.service  graphical.target.wants  sockets.target.wants
dbus-org.freedesktop.Avahi.service  halt.target.wants      sshd.service
dbus-org.freedesktop.ModemManager1.service  juiceshop.service      sysinit.target.wants
dbus-org.freedesktop.timesync1.service  multi-user.target.wants  syslog.service
default.target          network-online.target.wants  timers.target.wants
default.target.wants    ota.service
dev-serial1.device.wants  poweroff.target.wants

syslogic@raspberrypi:~ $ cat /etc/systemd/system/ota.service
[Unit]
Description=Checks for a newer upgrade package and applies it if available

[Service]
Type=simple
ExecStart=/opt/ota/ota.sh

[Install]
WantedBy=network-online.target
syslogic@raspberrypi:~ $ ls -la /opt/ota
total 16
drwxr-xr-x 2 root root 4096 Aug 30 23:24 .
drwxr-xr-x 4 root root 4096 Aug 30 19:12 ..
-rwxr-xr-x 1 root root 876 Aug 30 23:23 ota.sh
-rw-r--r-- 1 root root 6 Aug 30 19:17 VERSION
syslogic@raspberrypi:~ $

```

5. If you view the contents of the VERSION file (`cat /opt/ota/VERSION`), you will see that it contains a single line of text: **1.0.0**
6. Next, view the contents of the ota.sh script file in nano for better readability: `nano /opt/ota/ota.sh`

```

GNU nano 5.4 /opt/ota/ota.sh
#!/bin/bash
function version_gt() { test "$(echo "$@" | tr " " "\n" | sort -V | head -n 1)" != "$1"; }

currentversion=$(cat /opt/ota/VERSION)
latestversion=$(curl https://raw.githubusercontent.com/syslogicinc/iotworkshop/refs/heads/main/latestversion)
if version_gt "$latestversion" "$currentversion"; then
    echo "Upgrade package $latestversion available. Downloading and applying..."
    # Download upgrade package
    curl -k -o /tmp/ota-upgrade.zip "https://raw.githubusercontent.com/syslogicinc/iotworkshop/refs/heads/main/latestversion.zip"

    # Extract upgrade package
    if [ ! -d /tmp/ota ]; then
        mkdir /tmp/ota
    fi
    unzip /tmp/ota-upgrade.zip -d /tmp/ota

    # Execute upgrade.sh
    /bin/bash /tmp/ota/upgrade.sh

    echo "Upgrade package applied"
else
    echo "No newer upgrade package available"
fi

[ File '/opt/ota/ota.sh' is unwritable ]
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^J Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo

```

7. Let's walk through this code to try and understand what it is doing:

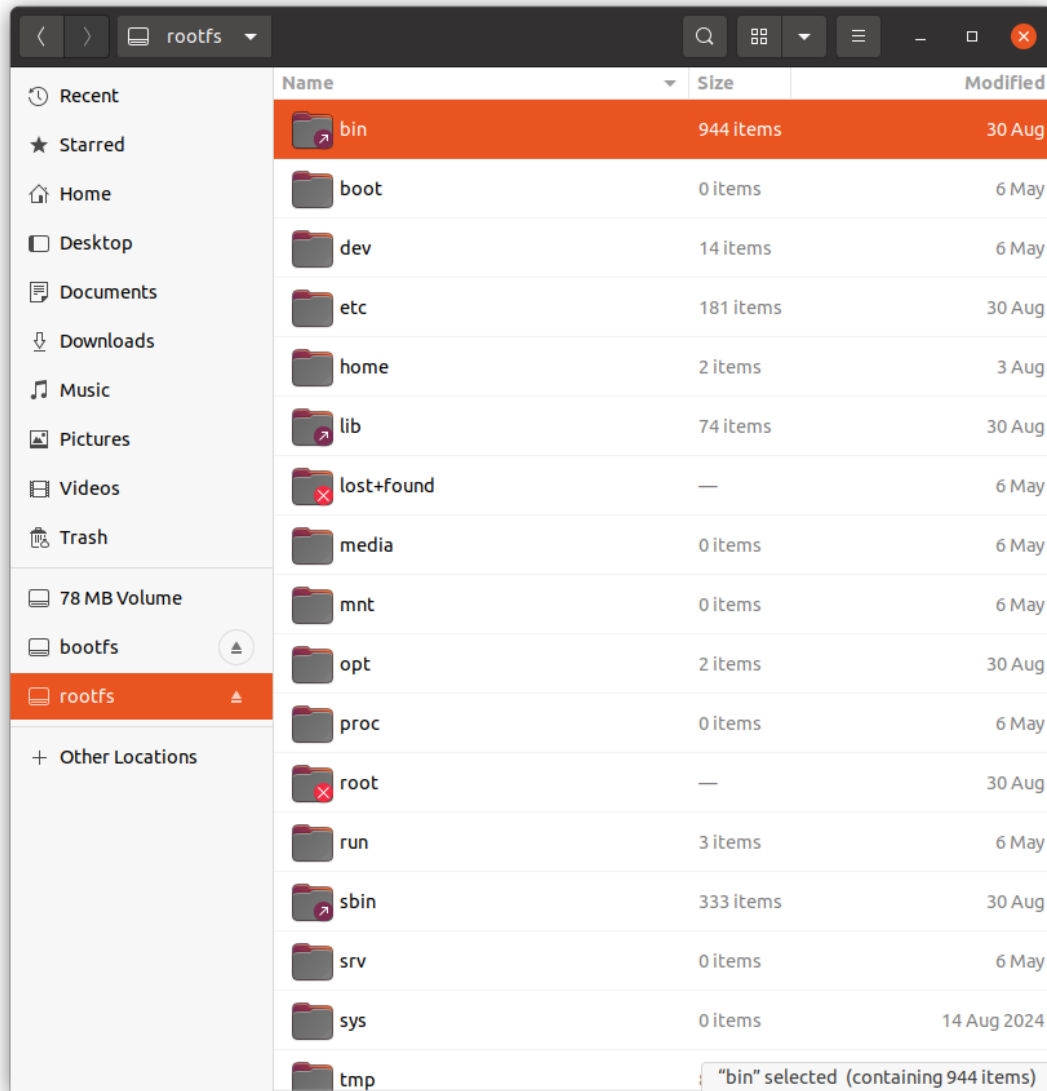
- a. A function is declared called "version_gt()" that appears to test if one version string is greater than another.
 - b. The variable "currentversion" is set to the contents of the /opt/ota/VERSION file (1.0.0)
 - c. The variable "latestversion" is set to a value retrieved from the URL:
<https://raw.githubusercontent.com/syslogicinc/iotworkshop/refs/heads/main/latest-version.json>
 - d. If latestversion is not greater than currentversion, then the script exits with the message "No newer upgrade package available"
 - e. Otherwise, it appears to download, extract, and execute an upgrade package from
[https://raw.githubusercontent.com/syslogicinc/iotworkshop/refs/heads/main/\\$latestversion.zip](https://raw.githubusercontent.com/syslogicinc/iotworkshop/refs/heads/main/$latestversion.zip)
8. There are several layers of protections typically employed in a secure OTA update process that we do not see in this implementation.
 - a. The connection to the internet hosted update files uses HTTPS, which is good, but there is no client authentication used.
 - b. There is no decryption of the upgrade package once downloaded. It is a simple .zip archive with no password protection.
 - c. There is no integrity verification of the upgrade package after it is downloaded. Ideally, this would take the form of a digital signature check.
9. Notice that the curl command to download the upgrade package includes the "-k" switch. What does this do?
 - a. See <https://linux.die.net/man/1/curl> for documentation on -k
 - b. Does this represent a security issue?

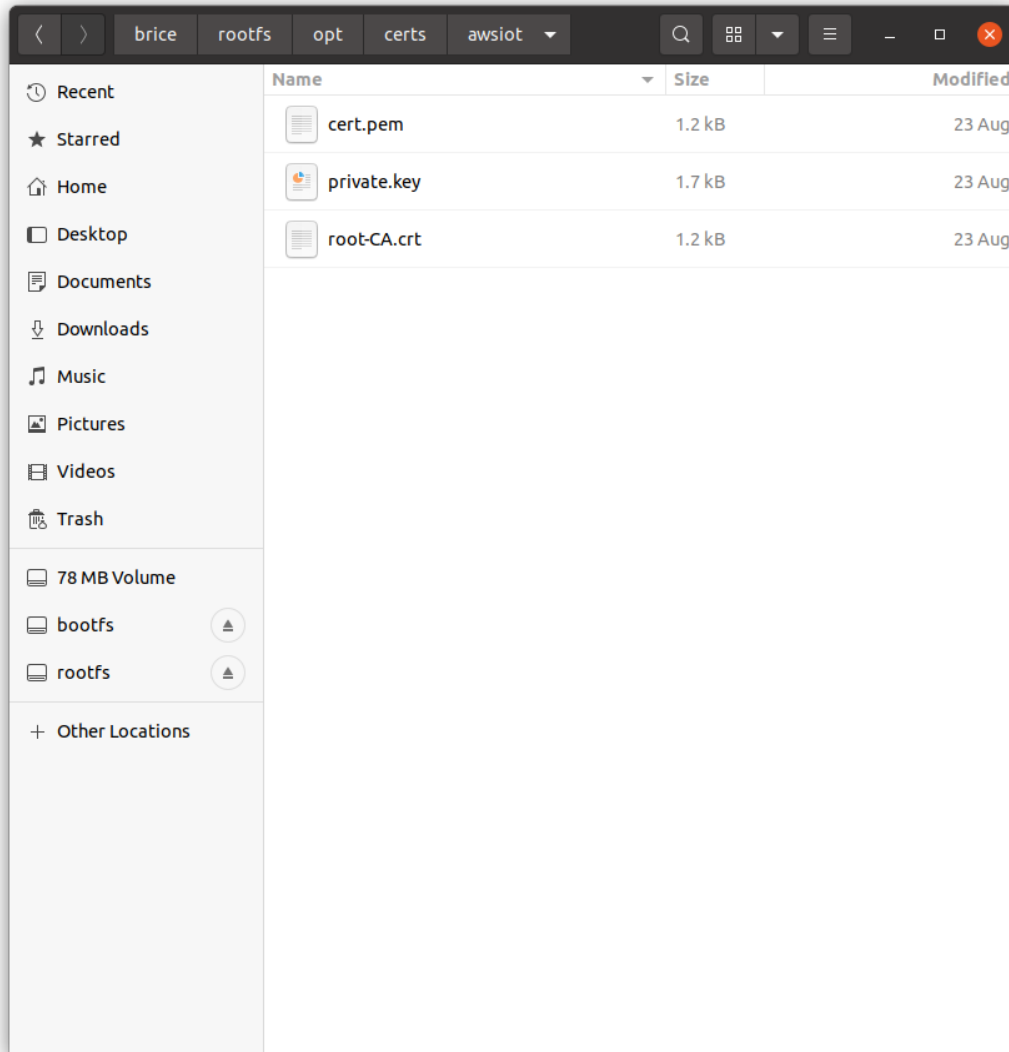
Exercise 9: Review Disk Encryption

1. Connect to the Raspberry Pi via SSH or the serial console cable.
2. Enter the following command to list information about all available block devices:
 - a. `lsblk`
3. You should see the output shown below. If any of the disk partitions are encrypted, you will see "crypt" in the TYPE column. Notice how neither of the two partitions shown are currently encrypted.

```
syslogic@raspberrypi:~ $ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
mmcblk0     179:0    0 29.1G  0 disk
├─mmcblk0p1 179:1    0 256M  0 part /boot
└─mmcblk0p2 179:2    0 28.9G  0 part /
syslogic@raspberrypi:~ $
```

4. No need to do this, but as an attacker if you were to remove the SD card from the Raspberry Pi and insert it into an SD card reader on another Mac or Linux computer you should be able to directly view and modify all files in the bootfs and rootfs partitions.

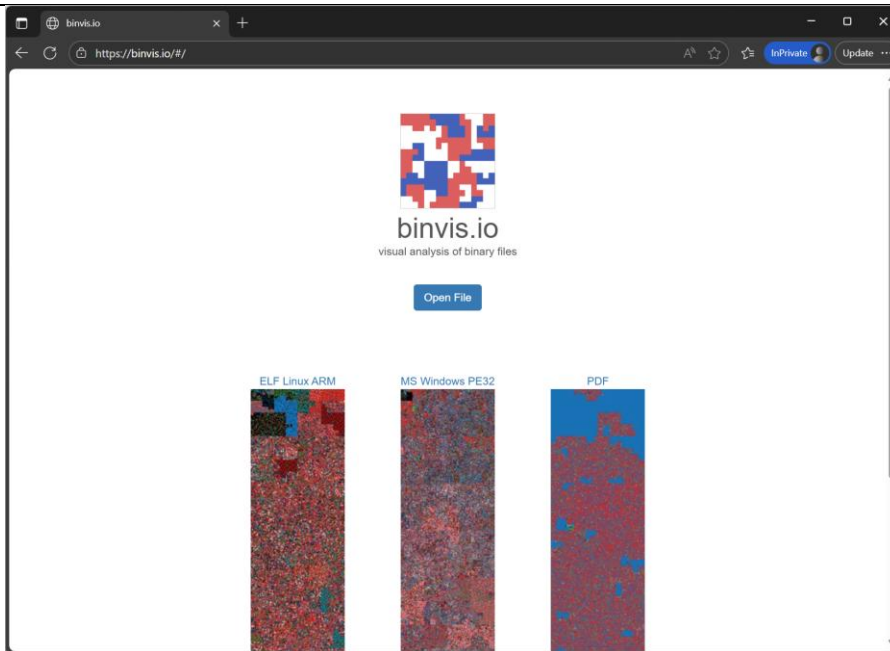




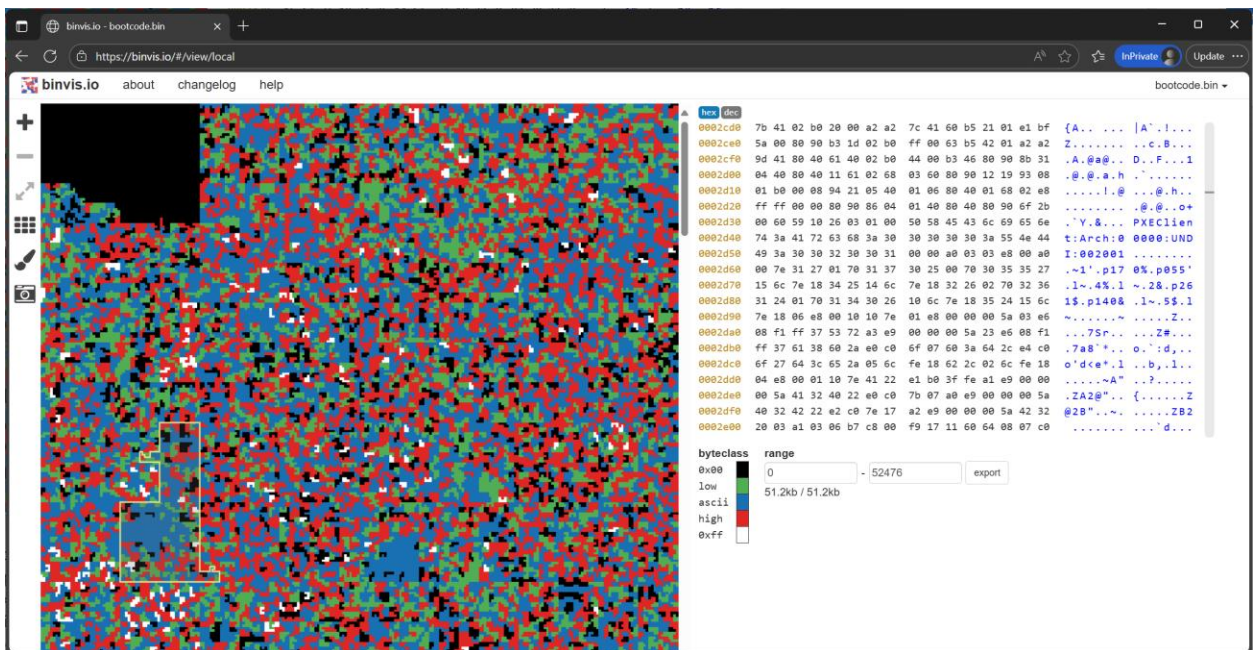
5. One way to protect the contents of the SD card is to implement disk encryption for the rootfs partition. See the instructions at <https://github.com/gitbls/sdm/blob/master/Docs/Disk-Encryption.md> for more information involving one technique. The SD content cannot be read, nor can the disk be booted, without knowing how to decrypt the partition either with a passphrase or a USB Keyfile Disk.

Exercise 10: Tamper with the Bootloader

2. If you have not already created a working folder as described in exercise #2, do so now.
3. Open a Command Prompt (Windows) or Terminal (Mac) window if not already open.
4. Run the following command to copy the bootcode.bin file from the Raspberry Pi to your working folder over SSH.
 - d. `scp syslogic@raspberrypi.local:/boot/bootcode.bin <working-folder>`
 - i. Replace <working-folder> with the actual path to your working folder.
 - e. You will be prompted to enter the password for the syslogic account (password123)
5. Open a browser and browse to <https://binvis.io>.



6. Upload the bootcode.bin file from your working folder using the **Open File** button.
7. You can visually tell that the file is not compressed or encrypted by the pattern displayed.



8. Click on various sections of the heat map and view data on the right. Observe that larger blue sections contain the most readable text. Can you find the reference to the **config.txt** file?
9. We won't actually be tampering with the bootcode.bin file with this exercise so as not to make the device inoperable, but you can see how easy it would be to do so using a standard binary editor like <https://mh-nexus.de/en/hxd/>.

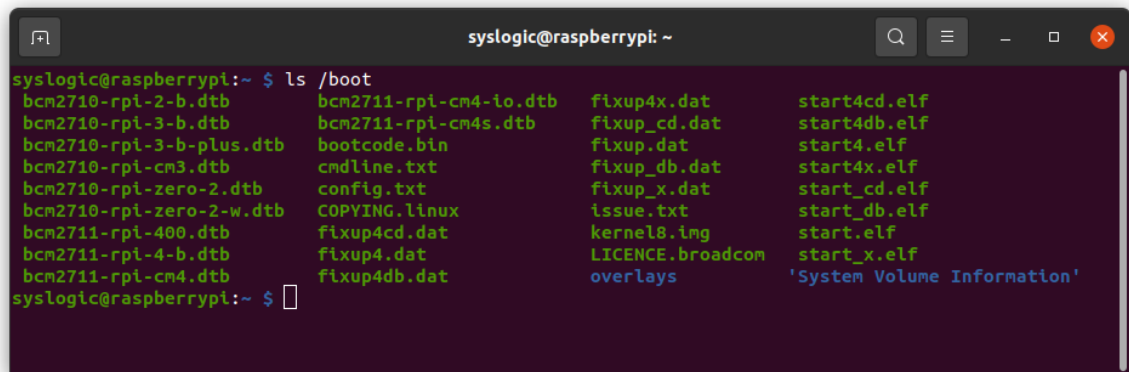
Raspberry Pi Boot Flow

This section is informational only and provides more detailed technical information about the Raspberry Pi's boot procedure.

1. There are 4-5 phases in the Raspberry Pi boot process (depending on the operating system used):
 - a. Phase 0: the mask ROM in the video processing unit (VPU) starts at power-up and will load bootcode.bin from one of many places (the RPI4 gets it exclusively from the EEPROM)
 - b. Phase 1: bootcode.bin will bring DRAM online, and load the start.elf file
 - c. Phase 2: start.elf will initialize most of the hardware, and load the kernel (either Linux itself, or u-boot in a zImage format)
 - d. Phase 3: the kernel will boot (which might be u-boot, and then loads Linux in a Phase 4)
2. The bootcode.bin file represents the bootloader code. Raspberry Pi versions 4 and 5 include an EEPROM memory chip for storage of this boot code. The security benefits of using EEPROM include:
 - a. EEPROM is non-volatile and not easily writable during normal operation. This makes it harder for attackers to modify the bootloader without physical access or special privileges.
 - b. You can lock down the EEPROM after flashing a secure bootloader, making it read-only. This prevents unauthorized updates or tampering with the boot process.
 - c. EEPROM-based bootloaders can be configured to verify digital signatures of the OS or firmware before loading. This ensures only trusted software is executed, protecting against rootkits or malicious firmware.
 - d. Raspberry Pi EEPROM bootloaders support recovery modes via GPIO or USB. This allows secure recovery without compromising the integrity of the bootloader itself.

Exercise 11: Counterfeiting the Device

1. Connect to the Raspberry Pi via SSH.
2. Run the following command to view the files in the /boot folder: `ls /boot`



```

syslogic@raspberrypi:~ $ ls /boot
bcm2710-rpi-2-b.dtb      bcm2711-rpi-cm4-io.dtb  fixup4x.dat            start4cd.elf
bcm2710-rpi-3-b.dtb      bcm2711-rpi-cm4s.dtb   fixup_cd.dat           start4db.elf
bcm2710-rpi-3-b-plus.dtb bootcode.bin            fixup.dat              start4.elf
bcm2710-rpi-cm3.dtb      cmdline.txt             fixup_db.dat           start4x.elf
bcm2710-rpi-zero-2.dtb   config.txt              fixup_x.dat            start_cd.elf
bcm2710-rpi-zero-2-w.dtb COPYING.linux           issue.txt              start_db.elf
bcm2711-rpi-400.dtb      flxup4cd.dat           kernel8.img            start.elf
bcm2711-rpi-4-b.dtb      flxup4.dat             LICENCE.broadcom       start_x.elf
bcm2711-rpi-cm4.dtb      flxup4db.dat           overlays                'System Volume Information'
syslogic@raspberrypi:~ $
  
```

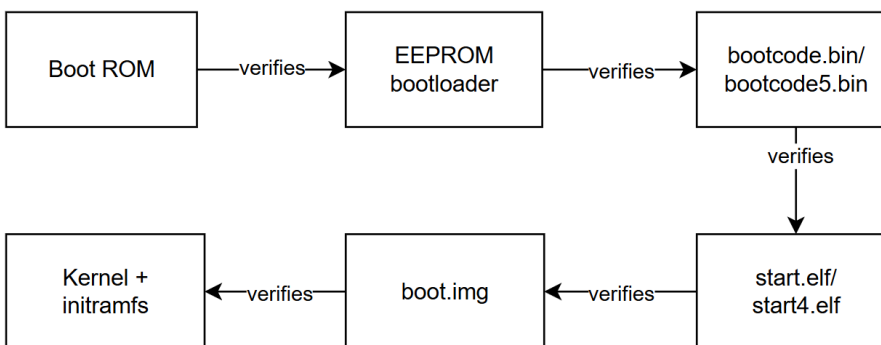
3. The bootfs partition is the first partition on the SD card and is mounted as /boot when the Raspberry Pi is running. It contains critical files such as:
 - a. bootcode.bin: The bootloader for older Raspberry Pi models.
 - b. start*.elf: Firmware files for initializing the GPU.
 - c. config.txt: Configuration file for system settings.
 - d. cmdline.txt: Linux kernel command-line parameters.
 - e. Kernel files (e.g., kernel8.img): The Linux kernel image.
 - f. Device Tree files (*.dtb): Hardware configuration files.
4. Enter the following command to power down the Raspberry Pi: `sudo shutdown now`
5. Disconnect the Raspberry Pi device from your laptop.
6. Remove the SD card and exchange it with another workshop attendee, or with one of the SysLogic trainers.
7. Reconnect the Raspberry Pi with the new SD card and connect to it again via SSH.

8. Observe that there is nothing currently preventing you (or an attacker) from using the SD card in any other Raspberry Pi Zero 2 W. There is also nothing preventing you from changing any of the files, including the boot configuration, kernel, device tree files, etc.
9. What are some methods you as a product manufacturer could use to prevent your Raspberry Pi based IoT devices from being copied and modified?
 - a. Use a read-only root filesystem. See <https://www.dzombak.com/blog/2024/03/running-a-raspberry-pi-with-a-read-only-root-filesystem/>
 - b. Use a "Secure Boot" mechanism. See **Secure Boot** section below for more information.
 - c. Use a third-party security module add-on. See **Security Modules** section below for more information.

Secure Boot

Secure Boot is a security feature used in IoT and embedded devices to help ensure only authorized firmware and operating systems can run on the device. It establishes a hardware root of trust and creates a cryptographically verified boot chain to prevent unauthorized code execution during the boot process.

Unfortunately, the Raspberry Pi Zero 2 W does not support secure boot, but Raspberry Pi 4 and 5 models do. See <https://pip.raspberrypi.com/categories/1260-security/documents/RP-003466-WP/Boot-Security-Howto.pdf> and <https://deepwiki.com/raspberrypi/usbbboot/5-secure-boot> for more information.



Security Modules

Some device platforms have built-in hardware support for secure boot (NVIDIA Jetson, NXP i.MX). Others like the Raspberry Pi Zero 2 W support adding this via a third-party security module like the [Zymbit ZYMKEY4i](#). This module adds a real time clock with battery, secure key storage, tamper detection, measured system identity, and support for file system encryption.



References

Prerequisites

Laptop running Windows or MacOS with at least one available USB-A port, Wi-Fi, and 1GB free disk space

- [Prolific PL2303 driver](#) *
- [Nmap](#) *
- [Wireshark](#) *
- [OWASP ZAP](#) (Windows: use the winget option to include JRE 17 if Java is not already installed) *
- Windows: [Putty](#) *
- Mac: [Minicom](#)

* Requires local administrator privileges to install